

Web Technologies Lab Manual

STUDENT DETAILS

Name : G. Raj Kumar

ID Number : N220556

Section : CSE-02

Date : 31-03-2026

GIT HUB DETAILS

GitHub User ID : raj-220556

Repository URL : https://github.com/raj-220556/WTLab_220556.git

SUBJECT DETAILS

Subject Name : Web Technologies Lab

Subject Code: 23CS2283

FACULTY DETAILS

Faculty Name : Srinivasu Ch ,Assistant Professor,RGUKT-Nuzvid

Department : CSE

INSTITUTION DETAILS

Rajiv Gandhi University of Knowledge Technologies – Nuzvid

ACADEMIC YEAR

2025–2026

Index / List of Experiments

S.No	Experiment Name	Date	Marks
1.	HTML and CSS		
2.	Advanced CSS		
3.	Java Script		
4.	Login and Signup using PHP		
5.	File Upload and Download System		
6.	O-Auth Integration		
7.	MongoDB integration		
8.	Git Industry Commands		
9.	JQuery Lab		
10.	Bootstrap		

Lab 1: HTML & CSS

Aim :

To implement and demonstrate a static user interface for an educational platform “SudoSchool” using HTML and CSS, and to manage the project using Git and GitHub.

Description

This experiment focuses on designing a user interface for an online learning platform. It involves creating structured web pages using HTML5 semantic elements and styling them using CSS. The project includes designing components such as a sticky top navigation bar, a horizontally scrollable course sub-navigation menu, a pricing table, and embedded multimedia elements (video, audio, iframes). It also emphasizes version control using Git and maintaining a proper repository on GitHub.

Task:

Develop a web application that demonstrates the following:

- Design a learning platform UI (SudoSchool).
- Create a homepage with header, sticky navigation, and sub-navigation menus.
- Implement a deeply structured course overview and pricing table.
- Utilize HTML5 ``<video>`` and ``<audio>`` tags for interactive media.
- Utilize ``<iframe>`` tags to securely embed external PDFs and websites.
- Use CSS for styling and layout (Grid system, sticky positioning, hover states).
- Maintain the project using Git step-by-step and push it to a GitHub repository.

Definitions :

- **HTML:** A markup language used to structure web page content.
- **CSS:** A stylesheet language used to style and design web pages.
- **Grid:** A CSS layout model used to define explicit rows and columns for structured alignment.
- **Iframe:** An HTML element used to embed another document or website within the current HTML document.

Code / Implementation :

HTML Code :-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>SudoSchool</title>
  <link rel = "stylesheet" href = "style.css">
</head>
<body>
  <header >
    <a href=/index.html></a>
    <input id="serach" type="search" placeholder=" Search..." style="border-radius:
30px;">
    </input>
    <button style="border-radius: 50px;"><img alt="search icon" data-bbox="500 820 520 840"/></button>
  <nav>
    <a href="/pages/about.html">About</a>
    <a href="#course">Courses</a>
    <a href="/pages/teachers.html">Teachers</a>
    <a href="#contact">Contact us</a>
  </nav>
</header>
<div>
  <div id="subnav">
```

```

        <a href="/Courses/html.html">HTML</a>
        <a href="/Courses/css.html">CSS</a>
        <a href="/Courses/html.html">PHP</a>
        <a href="/Courses/css.html">Git</a>
        <a href="/Courses/html.html">Javascript</a>
        <a href="/Courses/css.html">Jquery</a>
        <a href="/Courses/html.html">SQL</a>
        <a href="/Courses/css.html">MongoDB</a>
        <a href="/Courses/html.html">AI</a>
        <a href="/Courses/css.html">ML</a>
        <a href="/Courses/html.html">Datascience</a>
        <a href="/Courses/css.html">Cybersecurity</a>
    </div>
</div>
<div id="container">
    <div id="about">
        About
        <div><h2>Div1</h2></div>
        <h2>Div2</h2>
        <div>
        </div>
    </div>
    <div id="courses">
        Courses
    </div>
    <div id="teachers">
        Teachers
    </div>
    <div id="contact">
        Contact us
    </div>
</div>
<footer>
    &copy; Reserved All rights
</footer>
</body>
</html>

```

CSS:

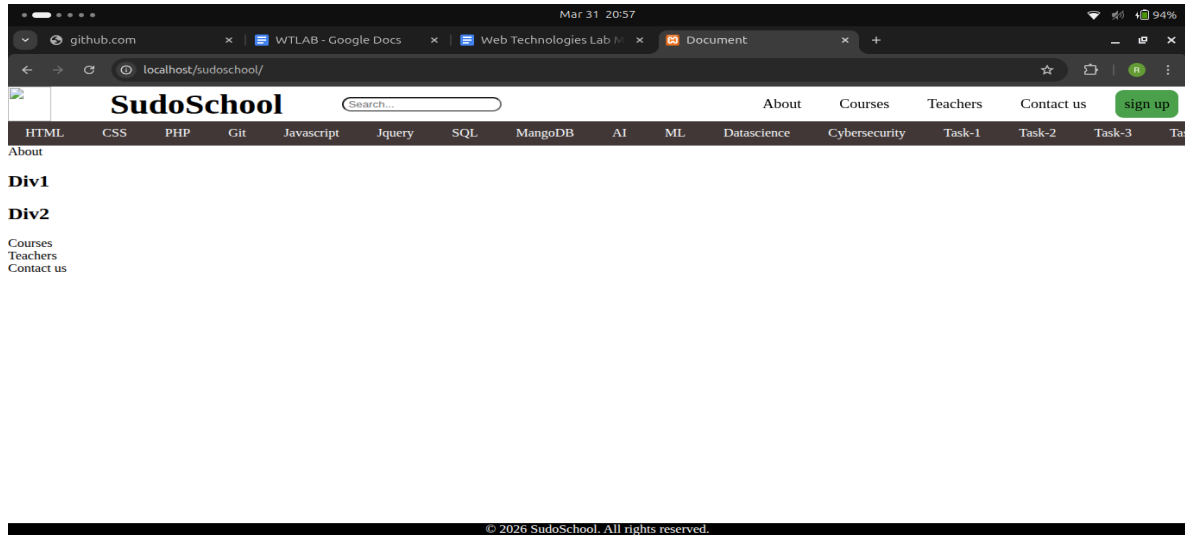
```

body{
    color: rgb(0, 0, 0);
    background-color: rgb(255, 255, 255);
    margin:0;
    padding:0;
    height: 100vh;
    display: grid;
    grid-template-columns: 1fr;
    grid-template-rows: 50px 35px 1fr 20px;
}
#footer{
    background-color: black;
    color: white;
    text-align: center;
    bottom: 0;
}
#logintag{
    padding-left: 18px;
    padding-right: 18px;
    color: white;
    background-color: rgb(78, 160, 78);
    border-radius: 10px;
}
#logintag:hover{
    background-color: rgb(3, 102, 3);
}
#nav{

```

```
background-color: rgb(255, 255, 255);
align-items: center;
display: flex;
color: black;
position: sticky;
top: 0;
margin: 0px;
padding: 0%;
padding-bottom: 0%;
padding-top: 0%;
}
#nav a{
color: black;
font-size: 18px;
margin: 10px;
padding: 10px;
text-decoration: none;
}
#nav a:hover{
background-color: rgba(206, 196, 196, 0.568);
border-radius: 20px;
}
#ss{
padding: 0px;
margin: 0;
margin-left: 5%;
margin-right: 5%;
font-size: 40px;
}
#subnav{
background-color: rgb(65, 57, 57);
position: sticky;
top: 50px;
margin-block: 0px;
overflow-x: auto; /* Enable horizontal scroll */
white-space: nowrap; /* Keep links in a single row */
scrollbar-width: none;
scrollbar-base-color: black;
left: 0;
right: 0;
}
#subnav a{
position: relative;
background-color: rgb(65, 57, 57);
color: rgb(253, 254, 255);
padding: 8px;
padding-left: 20px;
padding-right: 20px;
margin: 0px;
display: inline-block; /* Essential for padding/margin to work */
text-decoration: none;
}
#subnav a:hover{
background-color: rgb(48, 45, 45);
color: rgb(37, 199, 253);
}
```

Results / Screenshots:

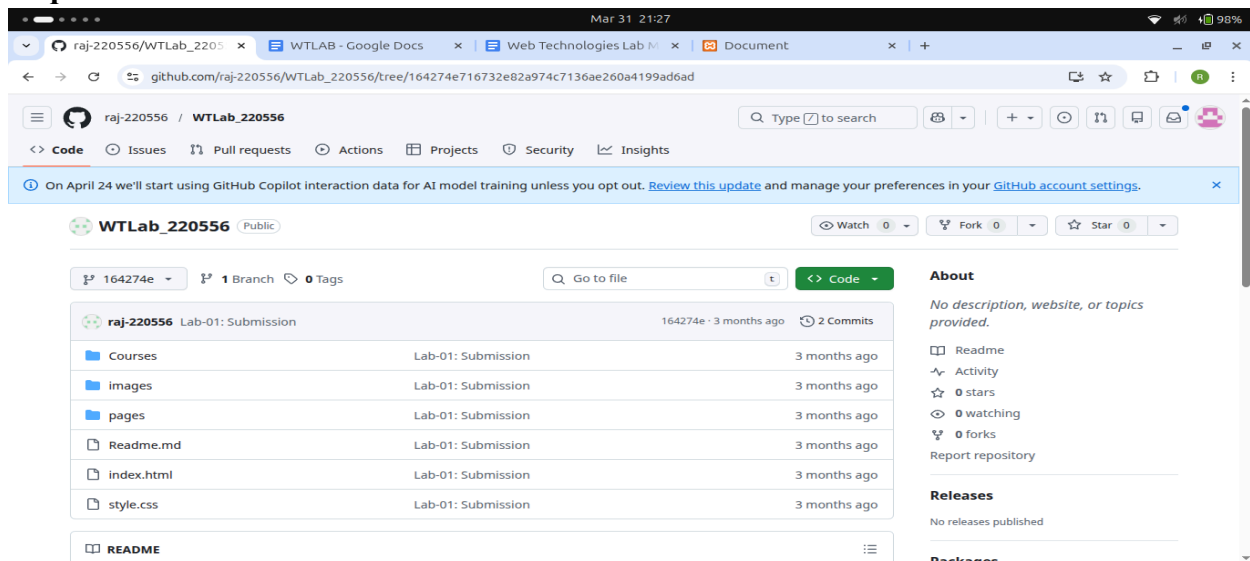


GitHub Repository Evidence:

Repository Details:

- **GitHub User ID:** raj-220556
- **Repository Name:** WTLab_220556
- **Repository URL:** https://github.com/raj-220556/WTLab_220556.git

Required Screenshots:



Conclusion

Successfully implemented an educational platform UI “SudoSchool” using comprehensive HTML and CSS and demonstrated version control tracking using Git.

Student Details :

Name: G. Raj Kumar

Roll Number: 31

Section: CSE-2

Date: 2026-03-31

Faculty Remarks :

Lab 2: Advanced CSS

Aim:

To enhance the user interface of the SudoSchool educational platform using advanced CSS techniques and to develop a professional, responsive, and industry-level web design.

Description

This experiment focuses on improving the basic structure created in Lab-1 by applying advanced CSS concepts. The SudoSchool website interface is enhanced using modern styling techniques such as Flexbox, CSS Grid, transitions, hover effects, and responsive design. The objective is to transform the basic layout into a visually appealing and professional user interface similar to real-world Education platforms.

Task:

Develop a web application that demonstrates the following:

- Enhance the existing MyTube UI using advanced CSS
- Apply Flexbox for alignment and layout design
- Use CSS Grid for structured content display (video section)
- Implement hover effects and transitions for better user experience
- Design a sticky navigation header
- Apply proper spacing using margin, padding, and box model
- Use Google Fonts for improved typography
- Create a responsive layout using media queries
- Maintain the project using Git and push updates to GitHub

Definitions:

- **Flexbox:** A CSS layout model used to align and distribute elements efficiently in a container.
- **CSS Grid:** A layout system used to create structured and responsive grid-based designs.
- **Box Model:** A CSS concept that includes margin, border, padding, and content for element spacing.
- **Media Queries:** CSS techniques used to make web pages responsive across different screen sizes.
- **Positioning:** A method used to control the placement of elements using properties like relative, absolute, and sticky.

Code / Implementation

HTML:

```
<div id="csscontainer">
  <div id="cssleft">
    <h3 style="font-size: 7rem;margin: 0;">CSS</h3>
    <p>The Language for Styling Web Pages</p>
    <a id="lcss" href="#">Learn CSS</a>
    <a id="cssref" href="#">CSS Reference</a>
    <a id="csscref" href="#">Get Certified</a>
  </div>
  <div id="cssright">
    <h3 style="font-size: 20px;">CSS Example: </h3>
    <div>
    </div>
    <a href="#">Try it Yourself</a>
  </div>
</div>
<div id="courses">
  <div id="php">
    <h1>PHP</h1>
    <h4>A web server programming language</h4>
    <a href="#">Learn PHP</a>
  </div>
```

```

<div id="jquery">
  <h1>jQuery</h1>
  <h4>A JS library for developing web pages</h4>
  <a href="#">Learn jQuery</a>
</div>
<div id="java">
  <h1>Java</h1>
  <h4>A programming language</h4>
  <a href="#">Learn Java</a>
</div>
<div id="c2">
  <h1>C++</h1>
  <h4>A programming language</h4>
  <a href="#">Learn C++</a>
</div>

```

CSS:

```

/* CSS Container */
#csscontainer{
  background-color: #96D4D4;
  color: black;
  height: 600px;
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 100px;
}
#cssleft{
  text-align: center;
  display: flex;
  align-items: center;
  flex-direction: column;
  width: 400px;
  height: 400px;
  margin: 30px;
}
#cssleft a{
  text-decoration: none;
  width: 150px;
  padding: 10px;
  margin: 10px;
}
#lcss{
  color: white;
  background-color: #04AA6D;
  border-radius: 20px;
}
#cssref{
  color: white;
  background-color: rgb(41, 37, 37);
  border-radius: 20px;
}
#cssright{
  color: black;
  background-color: rgb(218, 218, 218);
  width: 450px;
  height: 450px;
  padding: 20px;
  margin: 0px;
  margin-right: 70px;
  border-radius: 10px;
  box-shadow: 1px 1px 15px rgb(78, 76, 76);
}
#cssright a{
  text-decoration: none;
  background-color: #04AA6D;
  color: white;
  padding: 10px;
}

```

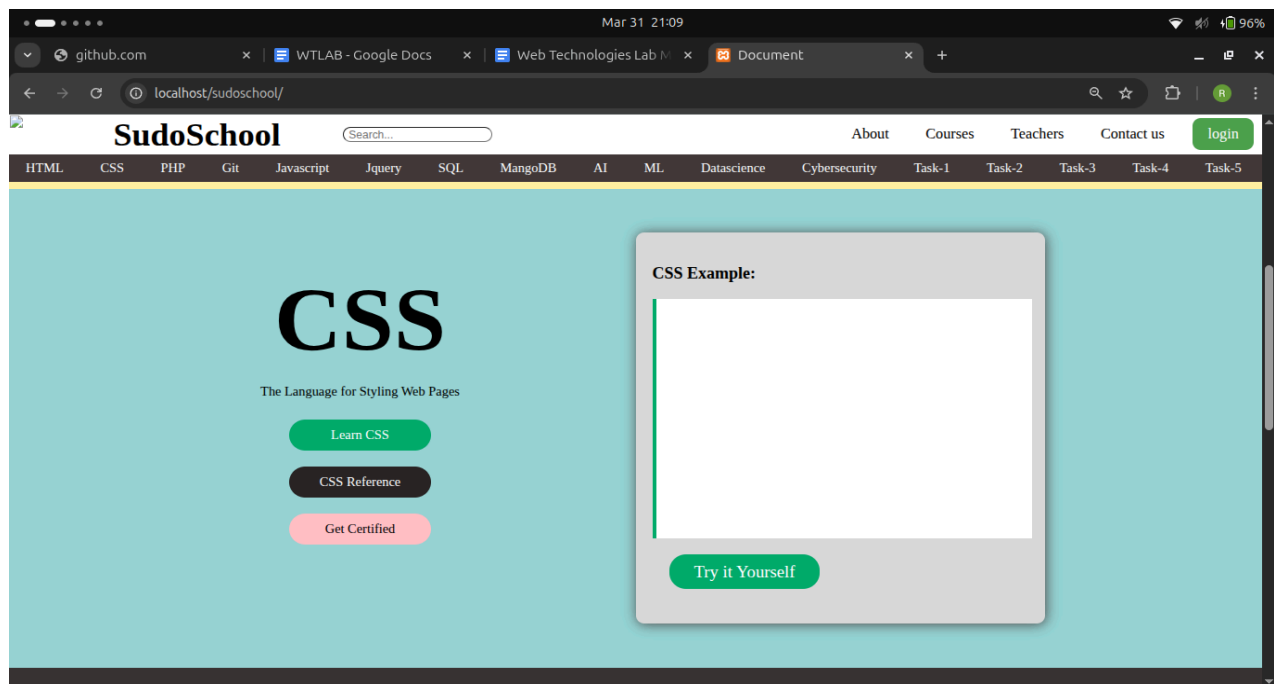


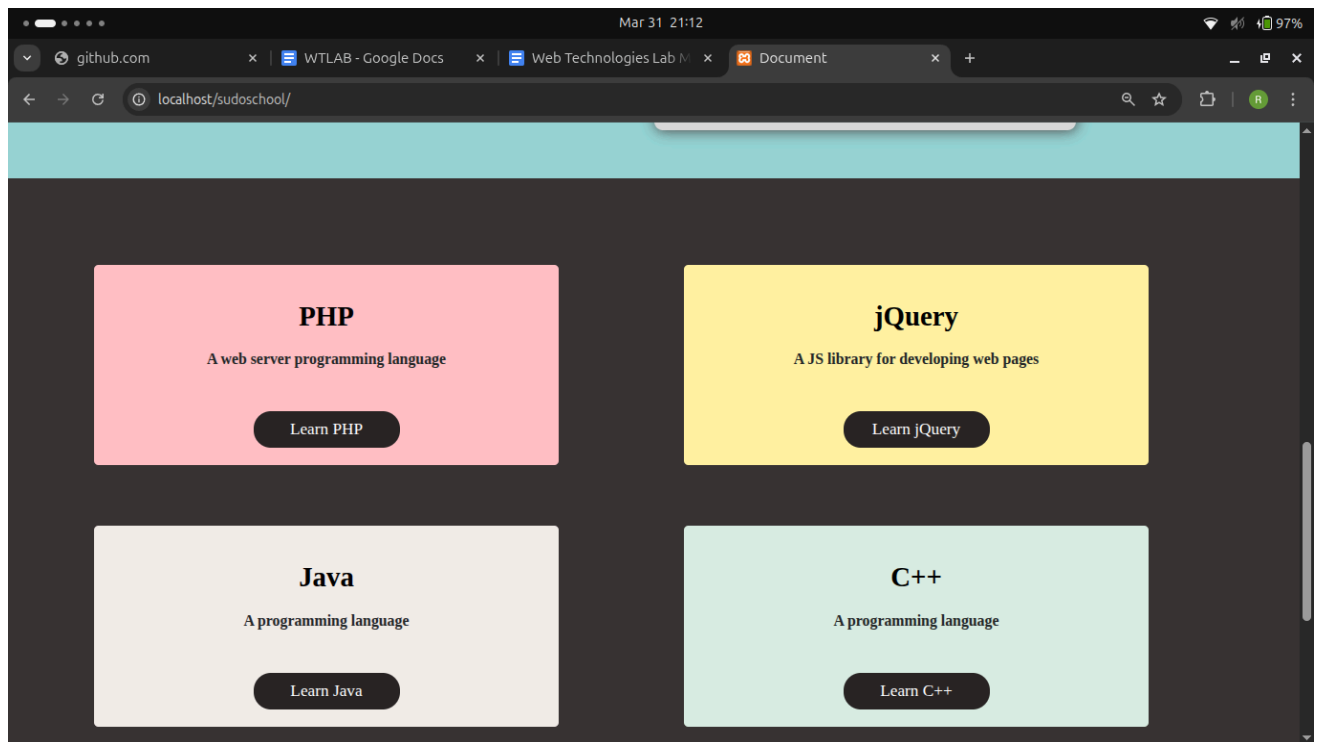
```

margin: 20px;
padding-inline: 30px;
border-radius: 20px;
}
#cssright a:hover{
background-color: #028656;
}
#courses{
background-color: rgb(56, 52, 52);
margin-top: 0px;
display: grid;
grid-template-columns: 1fr 1fr;
gap: 70px;
padding: 100px;
justify-content: center;
align-items: center;
}
#courses a{
background-color: rgb(41, 37, 37);
border-radius: 20px;
padding: 10px;
width: 150px;
text-align: center;
color: white;
text-decoration: none
}
#php{
background-color: #FFC0C7;
}
#jquery{
background-color: #FFF4A3;
}
#java{
background-color: #F3ECEA;;
}
#c2{
background-color: #D9EEE1;
}

```

Results / Screenshots





GitHub Repository Evidence:

Repository Details:

- **GitHub User ID:** raj-220556
- **Repository Name:** WTLab_220556
- **Repository URL:** https://github.com/raj-220556/WTLab_220556.git

Required Screenshots:

Commits on Dec 24, 2025

Lab-02: Adding CSS and register form added

raj-220556 committed on Dec 24, 2025

f5c388b <>

Conclusion

Successfully implemented an educational platform UI “SudoSchool” using comprehensive HTML and CSS and demonstrated version control tracking using Git.

Student Details :

Name: G. Raj Kumar

Roll Number: 31

Section: CSE-2

Date: 2026-03-31

Faculty Remarks :

Lab 3: Java Script

Aim:

To enhance the SudoSchool website by implementing interactive features using JavaScript concepts such as variables, functions, objects, methods, pop-up boxes, and event handling.

Description:

This experiment focuses on adding dynamic and interactive functionalities to the SudoSchool website using JavaScript. Various JavaScript concepts such as variables, functions, objects, methods, pop-up boxes, and event listeners are implemented to make the website responsive to user actions and improve user experience..

Task:

Develop a web application that demonstrates the following:

- Use variables to store and update dynamic data
- Implement different types of functions
- Create and use objects to manage data
- Define methods inside objects
- Use pop-up boxes (alert, confirm, prompt)
- Handle events using event listeners
- Dynamically update webpage content using JavaScript

Definitions:

- **Variable:** A container used to store data values in JavaScript.
- **Function:** A reusable block of code that performs a specific task.
- **Object:** A collection of key-value pairs used to represent real-world entities.
- **Method:** A function defined inside an object.
- **Event:** An action performed by the user (e.g., click, input).

Code / Implementation:

```
// variables
let x = 30;
const pi = 3.147;
var y =40;
console.log(x,y,pi);
document.write(pi,y,x);
// Normal declaration
function add(a,b){
    return a+b;
}
// Anonymous function declaration
const sub1 = function sub(a,b){
    return a-b;
}
console.log(sub1);
// Arrow Funtions
const mul = (a,b) => {
    return a*b;
};
// IIFE (Imediate Invoked Function Expression)
(function divs(){console.log(10/5);})();
console.log(add(2,3));
console.log(sub1(3,2));
console.log(mul(2,3));
// If else conditions
if(x>y){
    console.log(`${x} is greather than ${y}`);
}else if(y>x){
```

```
    console.log(`${y} is greather than ${x}`);
}else{
    console.log(`${x} and ${y} are Equal.`);
}
switch (x){
    case 10: console.log(x);
    default: console.log(y);
}
// Loops
// for -> for in(objects),for of(arrays)
for(let i=0; i<5; i++){
    console.log(i);
}
let list =[1,2,3,4,5]
for(l of list){
    console.log(l);
}
// while
let i =3;
while(i!=0){
    console.log(i);
    i--;
}
// do while
do{
    console.log(i);
}while(i!=0)
// Functions
function add2(...nums){
    let sum=0;
    for(num of nums){
        sum += num;
    }
    console.log(sum);
}
add2(1,2,3,4,5,6);
let name2 = "string";
let n = [...name2];
console.log(n);
console.log(name2.length);

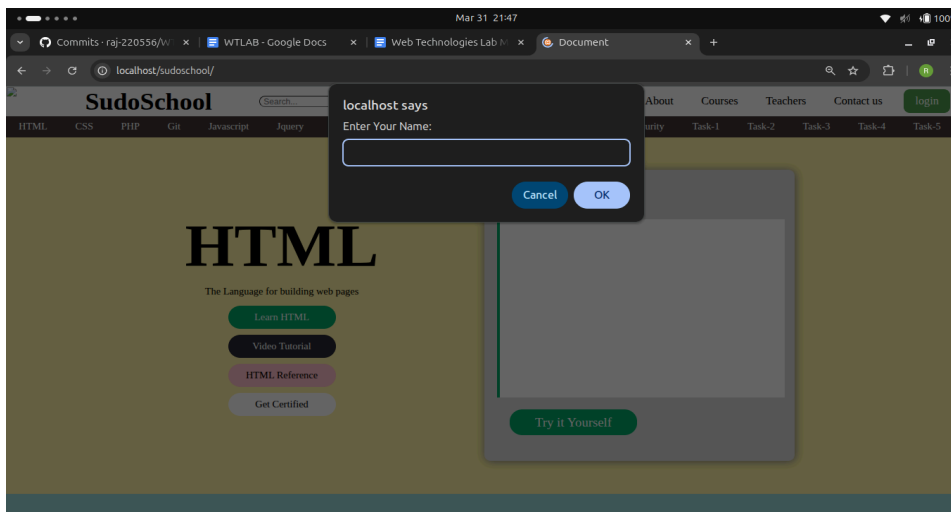
// Pop up boxs
// alert : just display msg
// window.alert(`hi hello`)
// prompt : it takes user input
let names = window.prompt("enter name:");
console.log(names);
// confirm : yes or no
if(window.confirm("hi")){
    console.log("true");
}else{object
    console.log("false");
}

// window.print() or clt+p :
window.print();
// DOM
const newh1 = document.createElement("h1");
newh1.style.backgroundColor = "tomato";
newh1.style.width = "100px";
newh1.style.height = "100px";
newh1.textContent = `h1`;

document.body.append(newh1);
document.body.prepend(newh1);
```

```
const box = document.getElementById("box");  
box.style.display = "none";  
console.log();
```

Results / Screenshots



GitHub Repository Evidence:

Repository Details:

- **GitHub User ID:** raj-220556
- **Repository Name:** WTLab_220556
- **Repository URL:** https://github.com/raj-220556/WTLab_220556.git

Required Screenshots:



Conclusion:

Successfully implemented JavaScript features including variables, functions, objects, methods, pop-up boxes, and event handling, resulting in an interactive and dynamic web application.

Student Details :

Name: G. Raj Kumar

Roll Number: 31

Section: CSE-2

Date: 2026-03-31

Faculty Remarks :

Lab 4: Login and Signup using PHP

Aim:

To develop a PHP-based login and registration system for the SudoSchool website using XAMPP and database integration, and to demonstrate PHP concepts such as variable scope, datatypes, and output functions.

Description

This experiment focuses on implementing backend functionality using PHP for user authentication. The SudoSchool website is enhanced by adding registration and login modules that interact with a database to store and validate user details. The experiment also demonstrates PHP concepts such as variable scope (local, global, static), datatypes, and output functions like echo, print, and die().

Task:

Develop a web application that demonstrates the following:

- Create a registration page to store user data
- Create a login page to validate user credentials
- Connect PHP with database (MySQL)
- Use POST method to send form data
- Test application using XAMPP
- Push the project to GitHub

Definitions:

- **PHP:** A server-side scripting language used to build dynamic web applications.
- **Variable Scope:** Defines where a variable can be accessed (local, global, static).
- **POST Method:** Used to send form data securely from client to server.
- **Database:** A structured system to store and manage data.
- **Session:** A way to store user data across multiple pages.

Code / Implementation:

DB.php:

```
<?php
    $db_hostname = "localhost";
    $db_user = "root";
    $db_database = "sudoschool";
    $db_password = "";
    $conn = mysqli_connect($db_hostname, $db_user, $db_password, $db_database);
    if(!$conn){
        die("Data Base Connection Failed <br>");
    }
?>
```

register.php:

```
<?php
    // DataBase Connection
    include("../..../database.php");
?>
<?php
    // Form Input Validation
    $flag = 1;
    if($_SERVER['REQUEST_METHOD'] == "POST"){
        if(empty($_POST["username"])){
            $flag = 0;
            die("Enter Username!! <br>");
        }
        if(empty($_POST["name"])){
            $flag = 0;
            die("Enter Name!! <br>");
        }
        if(empty($_POST["email"])){
            $flag = 0;
```

```

        die("Enter Email!! <br>");
    }
    if(empty($_POST["password"])){
        $flag = 0;
        die("Enter PAssword!! <br>");
    }
    if($flag){
        $username = trim(strtolower($_POST["username"]));
        $name = trim(ucwords(strtolower($_POST["name"])));
        $email = $_POST["email"];
        $password = $_POST["password"];
        if(strlen($password) < 8){
            die("Password Must conatin greather than 8 characters..");
        }
        if(!strpos($email, '@')){
            die("Enter a valid email");
        }
        $sql = "INSERT INTO users( username, name, password, email)
        VALUES ('$username', '$name', '$password', '$email')";
    }
}
echo "Request Method: " . $_SERVER['REQUEST_METHOD'] . "<br>";
foreach($_POST as $key => $value){
    echo "{$key} : {$value}";
}
if(empty($_POST)){
    echo "POST is Empty <br>";
}
if (mysqli_query($conn, $sql)){
    echo "Sucessfully Registered! <br>";
    foreach($_POST as $key => $value){
        echo "{$key} : {$value} <br>";
    }
}else{
    echo "Insertion Failed!";
}
}
?>

```

login.php:

```

<?php
    include "db.php";
    function loginUser(){
        global $conn;
        $username=$_POST['username'];
        $password=$_POST['password'];
        $username=strtolower(str_replace(' ','',$username));
        $sql="SELECT * FROM user_details WHERE username='$username' AND
        Password='$password'";
        $result=mysqli_query($conn,$sql);
        if(!$result){
            die("Query Failed");
        }
        if(mysqli_num_rows($result)==1){
            print"Login successful";
        }else{
            print"Invalid username or password";
        }
    }
    if($_SERVER["REQUEST_METHOD"]=="POST"){
        loginUser();
    }
}
?>

```

db_test.php:

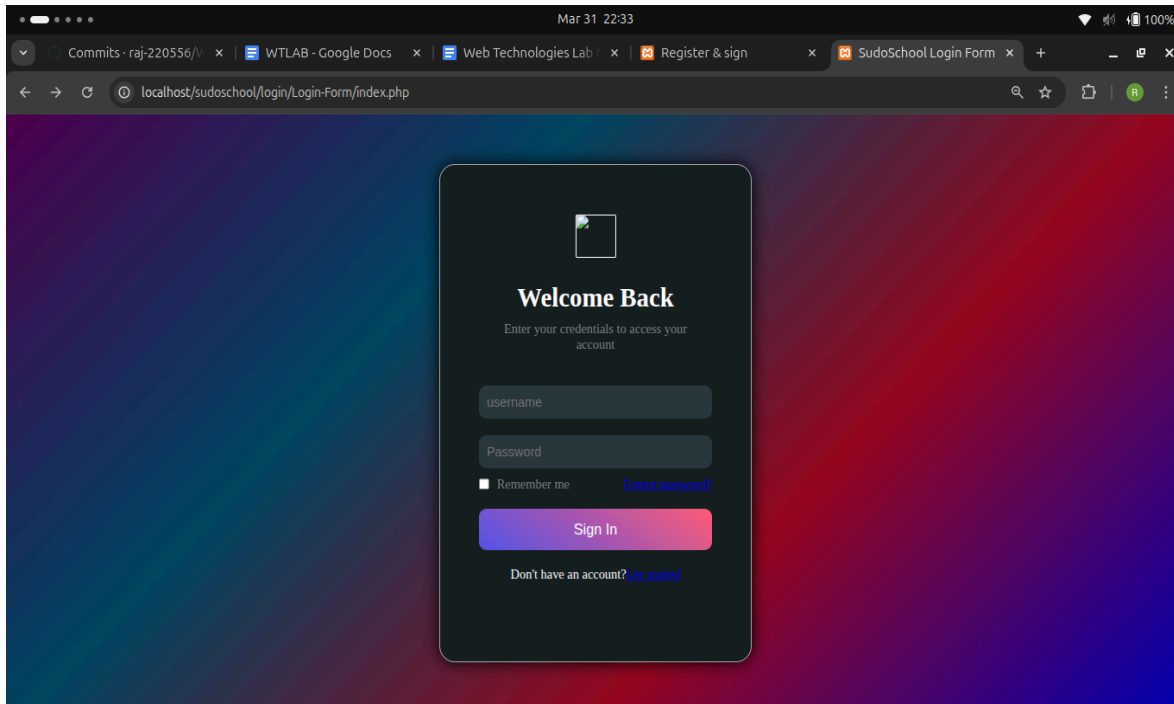
```

<?php
$conn=mysqli_connect("localhost","root","","testdb");
if($conn){

```

```
echo "Database Connected Successfully";  
}else{  
die("Connection Failed");  
}  
?>
```

Results / Screenshots



GitHub Repository Evidence:

Repository Details:

- **GitHub User ID:** raj-220556
- **Repository Name:** WTLab_220556
- **Repository URL:** https://github.com/raj-220556/WTLab_220556.git

Required Screenshots:



Conclusion

Successfully implemented a PHP-based login and registration system for the SudoSchool website using XAMPP and database integration, demonstrating variable scope, datatypes, and control functions with proper GitHub version control.

Student Details :

Name: G. Raj Kumar

Roll Number: 31

Section: CSE-2

Date: 2026-03-31

Faculty Remarks :

Lab 5: File Upload and Download System

Aim:

To implement file handling operations in PHP by developing a file upload and download system, and to demonstrate various PHP file functions and directory handling techniques.

Description:

This experiment focuses on implementing file handling concepts in PHP. A file upload and download system is developed where users can upload files, store them in a server directory, and download them later. Additionally, various PHP file functions are explored to read, write, manage, and retrieve file information, simulating a mini file manager system.

Task:

Develop a web application that demonstrates the following:

- Upload files using HTML form
- Store uploaded files in **uploads/** folder
- Provide download functionality
- Display file details (size, type, time)
- Perform file operations (create, read, delete, rename)
- Handle directories using PHP functions
- Demonstrate file modes using fopen()

Definitions

- **File Handling:** Process of creating, reading, writing, and managing files in PHP.
- **\$_FILES:** A PHP superglobal used to handle file uploads.
- **fopen():** Opens a file in a specified mode.
- **scandir():** Lists files in a directory.
- **unlink():** Deletes a file from the server.

Code / Implementation

upload.php:

```
<?php
    if ($_SERVER['REQUEST_METHOD'] == 'POST'){
        $temppath = $_FILES['file']['tmp_name'];
        $name = $_POST['filename'];
        $extension = '.txt';
        $destpath = 'uploads/' . $name . $extension;
        if (move_uploaded_file($temppath, $destpath)){
            echo "File Uploaded Sucessfull at $destpath <br> " ;
            $pwd = getcwd();
            echo "<a href = 'download.php?file=\"\". urlencode($destpath) . \"\"'> <button>
Download file </button> </a> ";
        }else{
            echo "something goes wrong";
        }
    }
?>
```

download.php:

```
<?php
    $file = $_GET['file'];
    if (file_exists($file)){
        header('Content-Description: File Transfer');
        header('Content-Type: application/octet-stream'); // Forces download for most types
        header('Content-Disposition: attachment; filename="' . basename($file) . '"');
        header('Expires: 0');
        header('Cache-Control: must-revalidate');
```

```

        header('Pragma: public');
        header('Content-Length: ' . filesize($file));
        flush();
        readfile($file);
        exit;
    }else{
        echo "File Not Found";
    }
}
?>

```

fileoperations.php:

```

<?php
$filename = 'uploads/test.txt';
$f = fopen($filename, 'a+');
/* FILE MODES DEFINITIONS:
    r - Read only (Starts at beginning)
    w - Write only (Truncates file to 0 length or creates if missing)
    a - Append (Starts at end of file; creates if missing)
    x - Create and open for write only (Returns FALSE if file exists)
    r+ - Read/Write (Starts at beginning)
    w+ - Read/Write (Truncates file or creates if missing)
    a+ - Read/Append (Starts at end; creates if missing)
    x+ - Create and open for Read/Write (Returns FALSE if file exists)
*/
if($f){
    // $text = fread($f, filesize($filename));
    // fseek($f, 0, SEEK_END);
    // echo $text . "<br>";

    $data = "\nThis is extra data into the file";
    // fwrite($f,$data);

    // rewind($f); // cursor to top position
    // clearstatcache();
    // $text2 = fread($f, filesize($filename));
    // echo $text2 . "<br>";

    // echo file_get_contents($filename);
    // echo file_put_contents($filename, $data);
    // echo file_get_contents($filename); //overwrites
    // print_r(file($filename));
    // echo file_exists($filename);
    // echo filesize($filename);
    // echo filetype($filename); //file
    // echo filetype("uploads"); //dir
    // echo fileatime($filename); // Access time (Fro 1970)
    // echo filemtime($filename); // Modification time
    // echo filectime($filename);
    // echo fileperms($filename);
    // echo fileowner($filename); //uid
    // echo filegroup($filename); //gid
    // echo fileinode($filename);

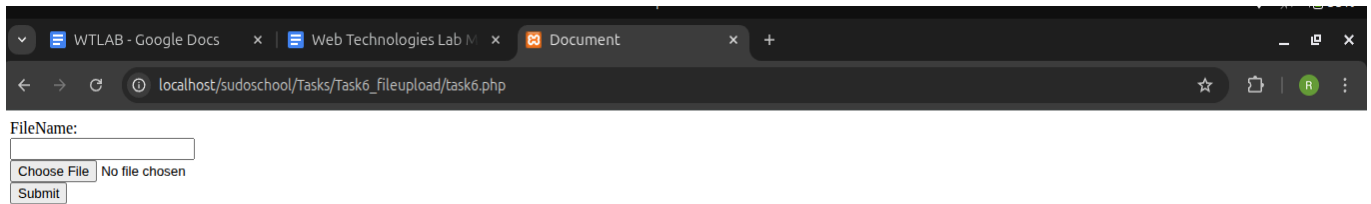
    // copy($filename, "new.txt");
    // rename('new.txt', "new2.txt");
    // unlink("new2.txt"); //deletes file
    // mkdir("newdir");
    // rmdir("newdir");
    // echo is_file($filename);
    // echo is_dir("uploads");

    // echo scandir("uploads");
    // $dir=opendir("uploads");
    // $data=readdir($dir);
    // closedir($dir);
    // echo getcwd();
    // flock($filename);

```

```
}else{  
    echo "File Error";  
}  
fclose($f);  
?>
```

Results / Screenshots

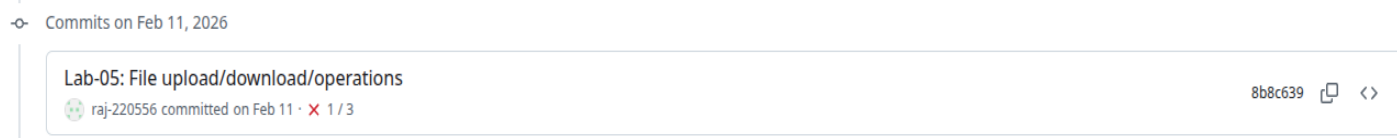


GitHub Repository Evidence:

Repository Details:

- **GitHub User ID:** raj-220556
- **Repository Name:** WTLab_220556
- **Repository URL:** https://github.com/raj-220556/WTLab_220556.git

Required Screenshots:



Conclusion

Successfully implemented Server-Side uploading logic and demonstrated version control using Git with proper repository management and commit history.

Student Details :

Name: G. Raj Kumar

Roll Number: 31

Section: CSE-2

Date: 2026-03-31

Faculty Remarks :

Lab 6: OAuth Integration

Aim:

To implement and demonstrate Secure OAuth 2.0 Integration (Google, Firebase, and GitHub/Facebook) using PHP and Firebase, while maintaining version control through Git and GitHub.

Description:

This experiment focuses on implementing external authentication providers to handle user identity without storing passwords locally. It utilizes Google Cloud Console for OAuth credentials, Firebase Authentication SDK for secondary support, and GitHub/Facebook Developer Portals for social login. The project emphasizes secure callback handling (Redirect URIs), environment variable management, and branding via CSS watermarking.

Task:

Develop a web application that demonstrates the following:

- Customer Support Portal: Integrated with Google OAuth using a PHP backend.
- Student Support Portal: Integrated with Firebase OAuth (supporting both client-side and server-side flows).
- Social Integration: Implementation of GitHub or Facebook Login.
- Branding: Custom logo combining User Name/ID and a CSS-based name watermark.
- Version Control: Full lifecycle management using Git with progressive commits.

Definitions:

- **OAuth 2.0:** An industry-standard protocol for authorization that allows a website to access data from another service without seeing user credentials.
- **Redirect URI:** The specific URL where the OAuth provider sends the user (and the authorization code) after they successfully log in.
- **Firebase Auth:** A service that provides backend services and easy-to-use SDKs to authenticate users to your app.
- **Client ID/Secret:** Unique identifiers used by the application to authenticate itself to the OAuth provider.

Code / Implementation:

google_oauth.php:

```
<?php
session_start();
require_once '../vendor/autoload.php';
try {
    $dotenv = Dotenv\Dotenv::createImmutable(__DIR__ . '/../..');
    $dotenv->load();
    $client = new Google\Client();
    $client->setClientId($_ENV['GOOGLE_CLIENT_ID']);
    $client->setClientSecret($_ENV['GOOGLE_CLIENT_SECRET']);
    $client->setRedirectUri($_ENV['GOOGLE_REDIRECT_URL']);
    $client->addScope("email");
    $client->addScope("profile");
    if (!isset($_GET['code'])) {
        $authUrl = $client->createAuthUrl();
        header('Location: ' . filter_var($authUrl, FILTER_SANITIZE_URL));
        exit();
    }
    $token = $client->fetchAccessTokenWithAuthCode($_GET['code']);
    if (isset($token['error'])) {
        throw new Exception('Error fetching token: ' . $token['error_description']);
    }
    $client->setAccessToken($token);
    $google_oauth = new Google\Service\Oauth2($client);
    $userInfo = $google_oauth->userinfo->get();
    $_SESSION['access_token'] = $token;
```

```

$_SESSION['user_email'] = $UserInfo->email;
$_SESSION['user_name'] = $UserInfo->name;
$_SESSION['user_picture'] = $UserInfo->picture;
header('Location: ../../index.php');
exit();
} catch (Exception $e) {
    echo 'Critical Error: ' . $e->getMessage();
}
?>

```

github_oauth.php:

```

<?php
require 'config.php';
session_start();
if (!isset($_GET['code'])) {
    die("Authorization failed.");
}
$ch = curl_init(TOKEN_URL);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
curl_setopt($ch, CURLOPT_POSTFIELDS, http_build_query([
    'client_id' => GITHUB_CLIENT_ID,
    'client_secret' => GITHUB_CLIENT_SECRET,
    'code' => $_GET['code'],
    'redirect_uri' => REDIRECT_URL
]));
curl_setopt($ch, CURLOPT_HTTPHEADER, ['Accept: application/json']);
$response = json_decode(curl_exec($ch), true);
$access_token = $response['access_token'];
$ch = curl_init(USER_URL);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
curl_setopt($ch, CURLOPT_HTTPHEADER, [
    'Authorization: Bearer ' . $access_token,
    'User-Agent: PHP-App' // GitHub requires a User-Agent header
]);
$user_data = json_decode(curl_exec($ch), true);
if (isset($user_data['login'])) {
    $_SESSION['user'] = [
        'username' => $user_data['login'],
        'avatar' => $user_data['avatar_url'],
        'name' => $user_data['name'] ?? $user_data['login'],
        'logged_in' => true
    ];
    header('Location: ../../index.php');
    exit();
}
?>

```

facebook_oauth.php:

```

<?php
require_once 'fb-config.php';
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}
try {
    $accessToken = $helper->getAccessToken();
} catch (Facebook\Exceptions\FacebookResponseException $e) {
    die('Graph returned an error: ' . $e->getMessage());
} catch (Facebook\Exceptions\FacebookSDKException $e) {
    die('Facebook SDK returned an error: ' . $e->getMessage());
}
if (!isset($accessToken)) {
    if ($helper->getError()) {
        header('HTTP/1.1 401 Unauthorized');
        echo "Error: " . $helper->getError() . "\n";
        echo "Error Code: " . $helper->getErrorCode() . "\n";
        echo "Reason: " . $helper->getErrorMessage() . "\n";
        echo "Description: " . $helper->getErrorDescription() . "\n";
    }
}

```

```

    } else {
        header('HTTP/1.1 400 Bad Request');
        echo 'Bad request - No token received.';
    }
    exit;
}

```

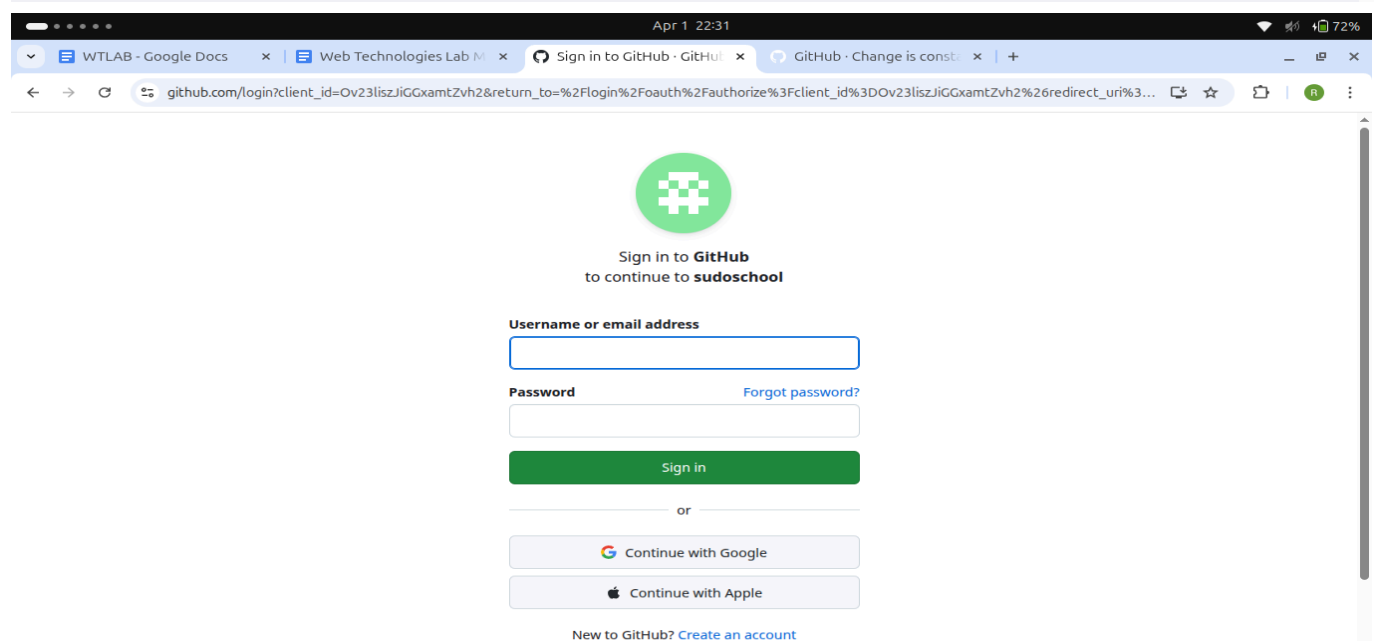
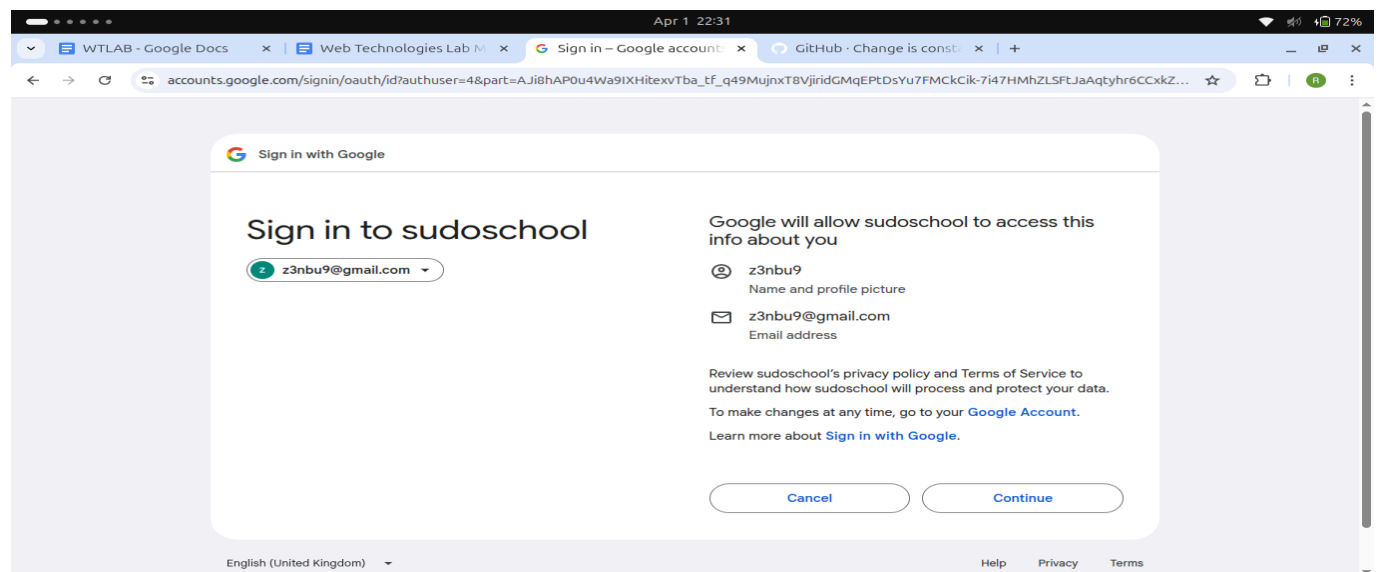
env:

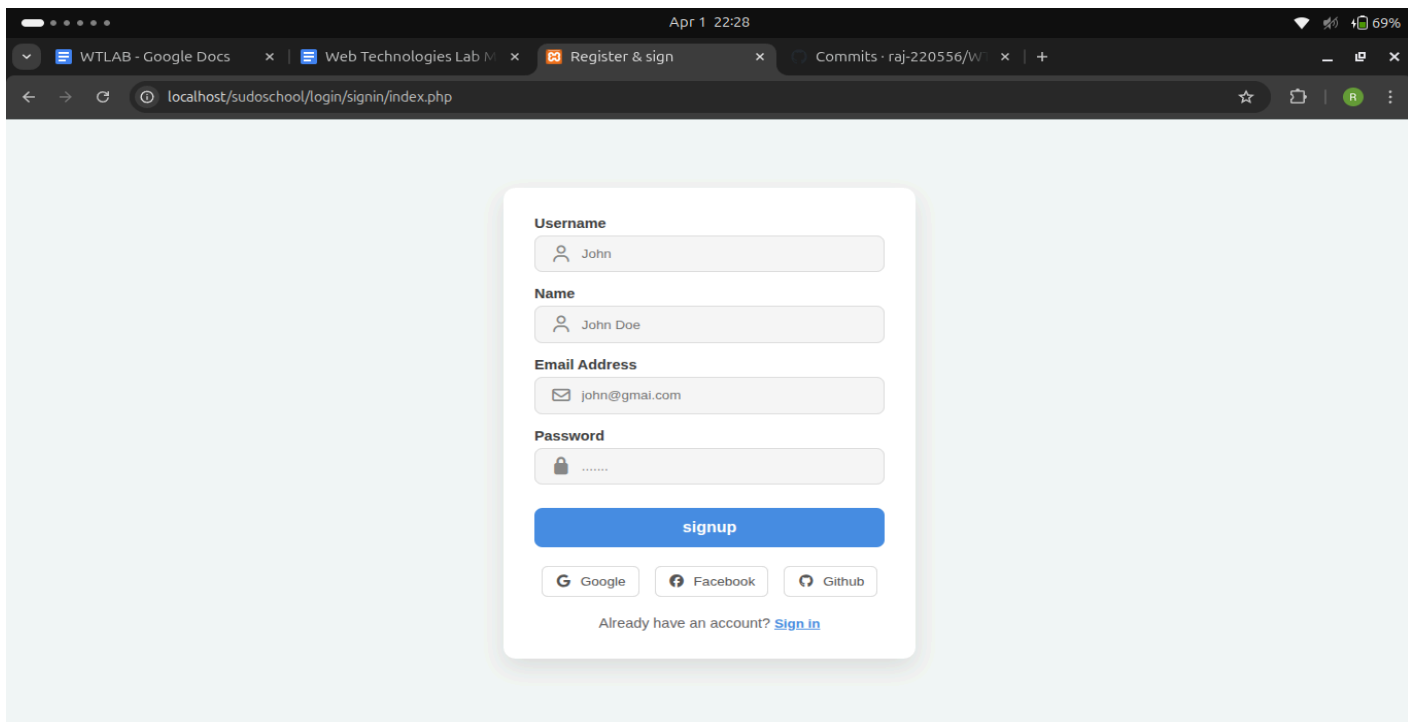
```

.env
1  # console.cloud.google.com
2  GOOGLE_CLIENT_ID=923540398998-fgthracndj1o5ehqrao7sd8egu106va0.apps.googleusercontent.com
3  GOOGLE_CLIENT_SECRET=G0CSPX-bS4UT84xXTIVaRQnZUwomyK6RAH5
4  GOOGLE_REDIRECT_URL=http://localhost/sudoschool/0Auth2.0/console/google_oauth.php
5
6  # Firebase
7  FIREBASE_DATABASE_URL=https://your-project-id-default-rtdb.firebaseio.com/
8  # Path to your JSON file (relative or absolute)
9  FIREBASE_SERVICE_ACCOUNT=service-account.json
10
11 #Github
12 GITHUB_CLIENT_ID=0v23liszJiGGxamtZvh2
13 GITHUB_CLIENT_SECRET=c31cd970bee6afd8debb853731410974a55b393a
14 GITHUB_REDIRECT_URL=http://localhost/sudoschool/0Auth2.0/github/callback.php
15
16 # Facebook
17 FB_APP_ID=1837588443606492
18 FB_APP_SECRET=b614e3800dba194f798e1217b9d8701a
19 FB_REDIRECT_URL=http://localhost/sudoschool/0Auth2.0/facebook/callback.php
20 # Firebase

```

Results / Screenshots:



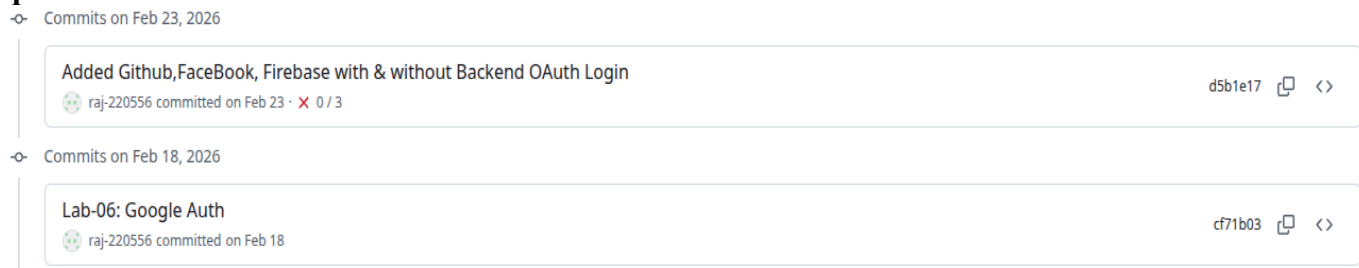


GitHub Repository Evidence:

Repository Details:

- **GitHub User ID:** raj-220556
- **Repository Name:** WTLab_220556
- **Repository URL:** https://github.com/raj-220556/WTLab_220556.git

Required Screenshots:



Conclusion

Successfully implemented Google, Firebase, and GitHub OAuth integrations and demonstrated version control using Git with proper repository management and a clear, progressive commit history.

Student Details :

Name: G. Raj Kumar

Roll Number: 31

Section: CSE-2

Date: 2026-03-31

Faculty Remarks :

Lab 7: MongoDB Integration

Aim:

To implement and demonstrate MongoDB Integration for a comprehensive User Authentication and Management System (Signup, Login, and CRUD) within the SudoSchool project using PHP and version control.

Description:

This experiment focuses on integrating a NoSQL database layer (MongoDB Community Server) into an existing web application. It covers establishing a database connection via a configuration file, implementing secure authentication workflows, and performing full CRUD (Create, Read, Update, Delete) operations. The implementation includes handling edge cases such as duplicate user registration, input validation, and secure session management, all while maintaining a step-by-step development history via Git.

Task:

Develop a web application for SudoSchool that demonstrates the following:

- **Database Connectivity:** Connect the backend to MongoDB using a dedicated configuration file and environment variables.
- **Authentication Module:** Create a Signup page to store user documents and a Login page to authenticate against MongoDB.
- **User Management:** Implement a dashboard to Read, Update (profile edits), and Delete user accounts.
- **Validation & Error Handling:** Prevent duplicate emails, manage empty fields, and handle database connection failures.
- **Project Structure:** Organize the project using a professional directory layout (/config, /models, /public).
- **Version Control:** Maintain the project using Git and push it to GitHub with a descriptive commit history.

Definitions

- **MongoDB:** A source-available, cross-platform, document-oriented NoSQL database program that uses JSON-like documents with optional schemas.
- **CRUD:** An acronym for Create, Read, Update, and Delete—the four basic functions of persistent storage.
- **Collection:** A grouping of MongoDB documents, equivalent to a "table" in relational databases.
- **Environment Variables (.env):** Variables whose values are set outside the program, typically used to store sensitive database credentials securely.

Code / Implementation

config/db.php:

```
<?php
require 'vendor/autoload.php';
try {
    $client = new MongoDB\Driver\Client("mongodb://localhost:27017");
    $db = $client->sudoschool_db;
    $usersCollection = $db->users;
} catch (Exception $e) {
    die("Database connection failed: " . $e->getMessage());
}
?>
```

signup.php:

```
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $email = $_POST['email'];
    $password = password_hash($_POST['password'], PASSWORD_DEFAULT);
    $existingUser = $usersCollection->findOne(['email' => $email]);
    if ($existingUser) {
        echo "Error: Email already registered.";
    } else {
```

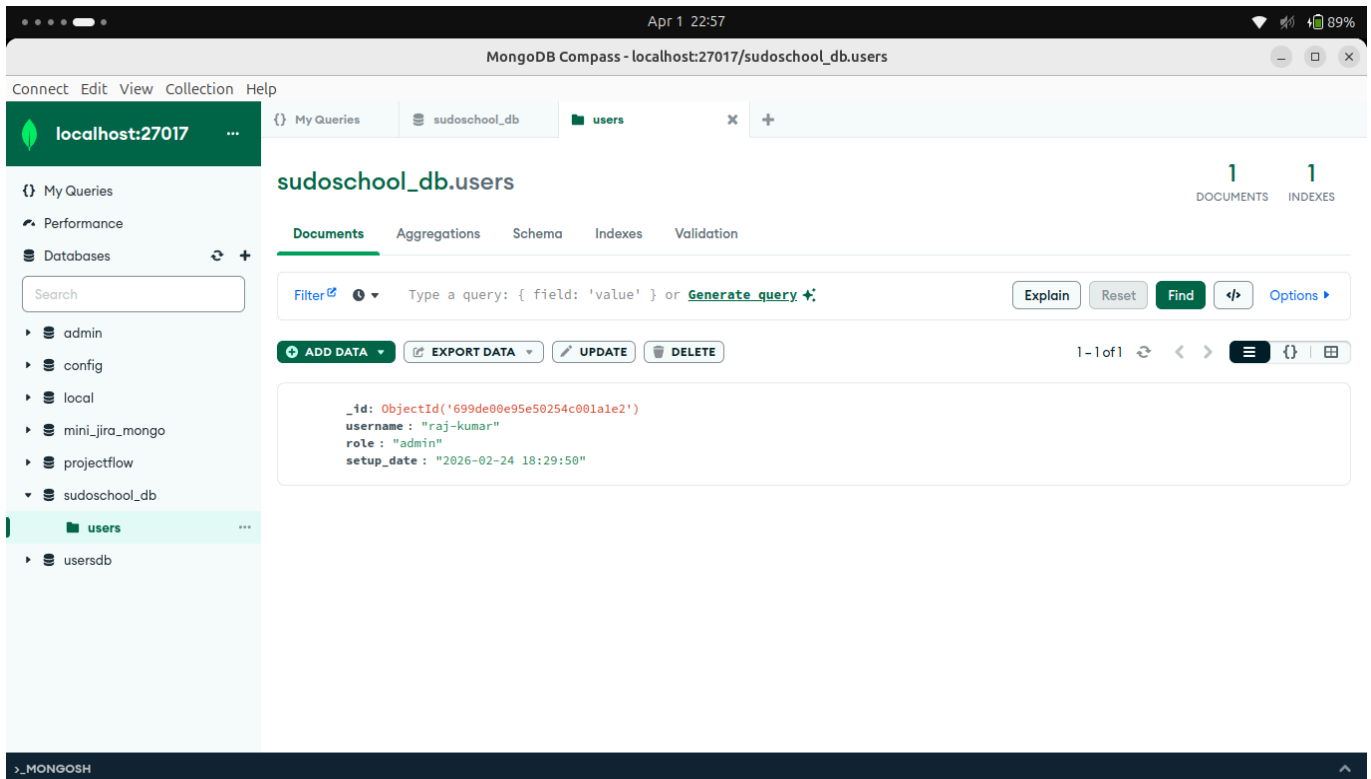


```

$insertResult = $usersCollection->insertOne([
    'username' => $_POST['username'],
    'email' => $email,
    'password' => $password,
    'created_at' => new MongoDB\BSON\UTCDateTime()
]);
echo "User registered successfully!";
}
}

```

Results / Screenshots



GitHub Repository Evidence:

Repository Details:

- **GitHub User ID:** raj-220556
- **Repository Name:** WTLab_220556
- **Repository URL:** https://github.com/raj-220556/WTLab_220556.git

Required Screenshots:



Conclusion

Successfully implemented Document Database handling mechanisms and demonstrated version control using Git with proper repository management.

Student Details :

Name: G. Raj Kumar
Roll Number: 31
Section: CSE-2
Date: 2026-03-31

Faculty Remarks :

Lab 8: Git Industry Commands

Aim:

To explore, execute, and document industry-level Git commands and GitHub workflows to maintain a professional version control environment for the SudoSchool project.

Description:

This experiment focuses on mastering advanced version control techniques including branching strategies, rebasing, stashing, and repository debugging. It also covers collaborative GitHub features such as Pull Requests (PRs), Issue Tracking, and Merge Conflict Resolution to simulate a real-world software development lifecycle (SDLC).

Task:

Develop a comprehensive documentation file and demonstrate the following:

- **Command Execution:** Execute 15 categories of Git commands (Configuration, Setup, Inspection, etc.).
- **Advanced Workflow:** Implement rebasing, cherry-picking, and stashing to manage complex code changes.
- **GitHub Integration:** Demonstrate repository management through Issues, Labels, Collaborators, and PR reviews.
- **Documentation:** Create `git_industry_commands.md` within the repository as a permanent reference.

Definitions:

- **Git Reflog:** A mechanism that records when the tips of branches and other references were updated in the local repository, allowing recovery of "lost" commits.
- **Cherry-pick:** The process of picking a specific commit from one branch and applying it onto another.
- **Rebase:** The process of moving or combining a sequence of commits to a new base commit to maintain a linear project history.
- **Stash:** A temporary storage area where you can "save" uncommitted changes to work on a different branch without losing progress.

Code / Implementation:

git_industry_commands.md:

Git Configuration Commands

1. ``git config --global user.name``

****syntax:****

````bash`

`git config --global user.name "Your Name"`

`````

****purpose:****

- Sets the name that will be attached to your commit transactions. This is how teammates identify who made specific changes in a shared repository.

****Example:****

````bash`

`git config --global user.name "raj-220556"`

`````

`![username](proofs/configname.png)`

2. ``git config --global user.email``

****syntax:****

````bash`

`git config --global user.email "yourname@example.com"`

`````

****purpose:****

- Associates your email address with your commits. Platforms like GitHub or GitLab use this email to link your local commits to your web profile.

****Example:****

```
```bash
git config --global user.email "n220556@rguktn.ac.in"
```
```

![email](proofs/configemail.png)

3. `git config --list`

****syntax:****

```
```bash
git config --list
```
```

****purpose:****

- Displays all the configuration settings Git can find at that moment, including your name, email, editor preferences, and alias settings. It's the "check your work" command.

****Example:****

```
```bash
git config --list
```
```

![conifg](proofs/configlist.png)

4. `git config --unset`

****syntax:****

```
```bash
git config --global --unset <key>
```
```

****purpose:****

- Removes a specific configuration setting. This is useful if you made a typo or want to revert a setting to its default state.

****Example:****

```
```bash
git config --global --unset user.name
```
```

![reoving configuration](proofs/configunset.png)

Repository Setup Commands

1. `git init`

****syntax:****

```
```bash
git init
```
```

****purpose:****

- Initializes a new, empty Git repository in the current directory.
- Creates the hidden `.git` folder required for tracking version history.
- Converts a standard folder into a workspace where you can start tracking code changes.

****Example:****

```
```bash
git init
```
```

2. `git clone`

****syntax:****

```
```bash
git clone <repository-url>
```
```

****purpose:****

- Downloads an existing repository from a remote server (like GitHub or GitLab) to your local machine.
- Automatically creates a directory with the project name and sets up the "origin" remote.
- Downloads the full history of the project by default.

****Example:****

```
```bash
git clone https://github.com/raj-220556/WT-demo_git.git
```
```

...

![cloning](proofs/clone.png)

3. `git clone --branch`

****syntax:****

```
```bash
git clone --branch <branch-name> <repository-url>
```
```

****purpose:****

- Clones the repository and immediately switches (checks out) to a specific branch.
- Ideal when you want to work on a "development" or "feature" branch rather than the default "main" branch.
- Saves you the extra step of running git checkout after cloning.

****Example:****

```
```bash
git clone --branch feature/frontend https://github.com/raj-220556/WT-demo_git.git
```
```

![clonig branch](proofs/configunset.png)

4. `git clone --depth`

****syntax:****

```
```bash
git clone --depth <number> <repository-url>
```
```

****purpose:****

- Performs a shallow clone by only downloading the latest commits (defined by the number).
- Greatly reduces download time and disk space for massive repositories where you don't need the full 10-year history.
- Often used in automation scripts or when you just need the current state of the code.

****Example:****

```
```bash
git clone --depth 1 https://github.com/raj-220556/WT-demo_git.git
```
```

Repository Status & Inspection

1. `git status`

****syntax:****

```
```bash
git status
```
```

****purpose:****

- Displays the state of the working directory and the staging area.
- Shows which changes have been staged, which haven't, and which files aren't being tracked by Git.

****Example:****

```
```bash
```

```
git status
```

```
```
```

![git status](proofs/clone.png)

2. `git log`

****syntax:****

```
```bash
```

```
git log
```

```
```
```

****purpose:****

- Shows the commit history for the current branch.
- Displays the SHA-1 hash, author, date, and the commit message.

****Example:****

```
```bash
```

```
git log
```

```
```
```

![logs of commits](proofs/gitlog.png)

3. `git log --oneline`

****syntax:****

```
```bash
```

```
git log --oneline
```

```
```
```

****purpose:****

- Condenses each commit into a single line.
- Useful for getting a high-level overview of the project history quickly.

****Example:****

```
```bash
```

```
git log --oneline
```

```
```
```

![logs are in single line](proofs/logoneline.png)

4. `git log --graph`

****syntax:****

```
```bash
```

```
git log --graph
```

```
```
```

****purpose:****

- Draws an ASCII graph representing the branch and merge history.
- Helps visualize how different branches have diverged and merged over time.

****Example:****

```
```bash
```

```
git log --graph
```

```
```
```

![reoving configuration](proofs/loggraph.png)

5. `git show`

****syntax:****

```
```bash
```

```
git show <commit-id>
```

```
```
```

****purpose:****

- Shows the metadata and content changes (the "diff") of a specific object, usually a commit.

****Example:****

```
```bash
git show d5b1e17
```
```

![reoving configuration](proofs/gitshow.png)

6. `git diff`

****syntax:****

```
```bash
git diff
```
```

****purpose:****

- Shows the changes between your working directory and the index (staging area).
- Specifically, it shows what you have modified but not yet added for the next commit.
- Simply added data and not added data. It shows data that which are not added in staging area

****Example:****

```
```bash
git diff
```
```

![reoving configuration](proofs/gitdiff.png)

7. `git diff --staged`

****syntax:****

```
```bash
git diff --staged
```
```

****purpose:****

- Shows the changes between the ****staging area** and the last commit****** (HEAD).
- It highlights what is about to be committed.

****Example:****

```
```bash
git diff --staged
```
```

![reoving configuration](proofs/gitdiffstaged.png)

8. `git blame`

****syntax:****

```
```bash
git blame <file-name>
```
```

****purpose:****

- Displays the last modification (author and commit) for each line of a file.
- Used to track down who made specific changes in a collaborative environment.

****Example:****

```
```bash
git blame index.php
```
```

![reoving configuration](proofs/gitblameindex.png)

9. `git reflog`

****syntax:****

```
```bash
git reflog
```
```

****purpose:****

- Records every time the tip of a branch is updated (e.g., switches, commits, resets).
- Acts as a safety net to recover "lost" commits that aren't visible in standard logs.

****Example:****

```
```bash
```

```
git reflog
```

```
```
```

![reoving configuration](proofs/gitreflog.png)

10. `git shortlog`

****syntax:****

```
```bash
```

```
git shortlog
```

```
```
```

****purpose:****

- Summarizes the git log output by grouping commits by author.
- Primarily used for creating release notes or checking contributor activity.

****Example:****

```
```bash
```

```
git shortlog
```

```
```
```

![reoving configuration](proofs/gitshortlog.png)

File Tracking Commands

1. `git add`

****syntax:****

```
```bash
```

```
git add <file-name>
```

```
```
```

****purpose:****

- Adds a specific file to the Staging Area (index).
- Tells Git that you want to include updates in this specific file in the next commit.

****Example:****

```
```bash
```

```
git add index.php
```

```
```
```

- changes or new file added into staging area

2. `git add .`

****syntax:****

```
```bash
```

```
git add .
```

```
```
```

****purpose:****

- Stages all changes in the current directory and its subdirectories.
- Includes new files, modified files, and deleted files.

****Example:****

```
```bash
```

```
git add .
```

```
```
```

3. `git add -p`

****syntax:****

```
```bash
```

```
git add -p
```

```
```
```

****purpose:****

- Opens patch mode, allowing you to interactively review and stage specific "hunks" (parts) of a file.
- Excellent for keeping commits clean when you've made multiple unrelated changes in a single

file.

****Example:****

```
```bash
```

```
git add -p
```

```
```
```

```
![reoving configuration](proofs/gitaddp.png)
```

4. `git restore`

****syntax:****

```
```bash
```

```
git restore <file-name>
```

```
```
```

****purpose:****

- Discards local changes in the working directory.
- Reverts a file back to the state it was in at the last commit. Use with caution, as unstaged changes will be lost.

****Example:****

```
```bash
```

```
git restore index.php
```

```
```
```

- Removes the data that difference of working directory & staging area in working directory.

5. `git restore --staged`

****syntax:****

```
```bash
```

```
git restore --staged <file-name>
```

```
```
```

****purpose:****

- Unstages a file.
- Removes the file from the Staging Area but keeps your actual code changes in the working directory.
- Useful if you accidentally ran git add on the wrong file.

****Example:****

```
```bash
```

```
git restore --staged index.php
```

```
git status
```

```
```
```

```
![restore staged](proofs/restorestaged.png)
```

6. `git rm`

****syntax:****

```
```bash
```

```
git rm <file-name>
```

```
```
```

****purpose:****

- Removes a file from both the working directory and the Git index(staging area).
- It stages the deletion, so the file will no longer exist in the next commit.

****Example:****

```
```bash
```

```
git rm test.txt
```

```
```
```

7. `git mv`

****syntax:****

```
```bash
```

```
git mv <old-name> <new-name>
```

```
```
```


****purpose:****

- Renames or moves a file and automatically stages the change.
- It is a shortcut for renaming the file manually and then running `git rm` on the old name and `git add` on the new one.

****Example:****

```
```bash
git mv old.txt new.txt
```
```

5. Commit Commands

git commit

Syntax

```
git commit
```

Purpose

Records changes to the repository and opens a text editor for commit message.

```
![git commit](./proof/img.png(48).png>)
```

git commit -m

Syntax

```
```bash
git commit -m "commit message"
```
```

Purpose

Creates a commit with a message directly from the terminal.

```
![git commit m](./proof/img.png(49).png>)
```

git commit --amend

Syntax

```
```bash
git commit --amend
```
```

Purpose

Modifies the most recent commit.

```
![git commit amend](./proof/img.png(50).png>)
```

git commit --no-edit

Syntax

```
```bash
git commit --amend --no-edit
```
```

Purpose

Amends the last commit without changing the commit message.

```
![git commit no edit](./proof/img.png(51).png>)
```

6. Branch Management Commands

git branch

Syntax

```
```bash
git branch
```
```

Purpose

Lists all local branches in the repository.

```
![git branch](./proof/img.png)

## git branch -a

### Syntax
```bash
git branch -a
```

### Purpose
Displays both local and remote branches.
![git branch a](./proof/img.png)

## git branch -d

### Syntax
```bash
git branch -d <branch-name>
```

### Purpose
Deletes a branch safely.
![git branch d](./proof/img.png)

## git branch -D

### Syntax
```bash
git branch -D <branch-name>
```

### Purpose
Force deletes a branch.
![git branch D](./proof/img.png)

## git checkout

### Syntax
```bash
git checkout <branch-name>
```

### Purpose
Switches to another branch.
![git checkout](./proof/img.png)

## git checkout -b

### Syntax
```bash
git checkout -b <branch-name>
```

### Purpose
Creates a new branch and switches to it.
![git checkout b](./proof/img.png)

## git switch

### Syntax
```bash
git switch <branch-name>
```

### Purpose
Switches to another branch.
```

```
![git switch](./proof/img.png)
```

```
## git switch -c
```

```
### Syntax
```

```
```bash
```

```
git switch -c <branch-name>
```

```
```
```

```
### Purpose
```

Creates and switches to a new branch.

```
![git switch c](./proof/img.png)
```

7. Merge & Integration Commands

```
## git merge
```

```
### Syntax
```

```
```bash
```

```
git merge <branch-name>
```

```
```
```

```
### Purpose
```

Combines changes from another branch into the current branch.

```
![git merge](./proof/img.png)
```

```
## git merge --no-ff
```

```
### Syntax
```

```
```bash
```

```
git merge --no-ff <branch-name>
```

```
```
```

```
### Purpose
```

Forces a merge commit even when fast-forward is possible.

```
![git merge no ff](./proof/img.png)
```

8. Remote Repository Commands

```
## git remote
```

```
### Syntax
```

```
```bash
```

```
git remote
```

```
```
```

```
### Purpose
```

Lists the names of remote repositories connected to the local repository.

```
### Example
```

```
```bash
```

```
git remote
```

```
```
```

```
![git remote](./proof/img.png)
```

```
## git remote -v
```

```
### Syntax
```

```
```bash
```

```
git remote -v
```

```
```
```

```
### Purpose
```

Displays remote repository URLs for fetch and push operations.

```
### Example
```

```
```bash
```

```
git remote -v
```
![[git remote -v](./proof/img.png)]

## git remote add

### Syntax
```bash
git remote add origin <repository-url>
```

### Purpose
Adds a remote repository connection to the local project.

### Example
```bash
git remote add origin https://github.com/raj-220556/demo-git.git
```
![[git remote add](./proof/img.png(64).png)]

## git remote remove

### Syntax
```bash
git remote remove origin
```

### Purpose
Removes a remote repository from the project.

### Example
```bash
git remote remove origin
```
![[git remote remove](./proof/img.png(65).png)]

## git fetch

### Syntax
```bash
git fetch
```

### Purpose
Downloads commits and objects from the remote repository without merging them.

### Example
```bash
git fetch origin
```
![[git fetch](./proof/img.png(66).png)]

## git fetch --all

### Syntax
```bash
git fetch --all
```

### Purpose
Fetches updates from all configured remote repositories.

### Example
```bash
git fetch --all
```
![[git fetch all](./proof/img.png)]

## git pull
```

```
### Syntax
```bash
git pull
```

### Purpose
Fetches and merges changes from the remote repository into the current branch.

### Example
```bash
git pull origin main
```

![[git pull](./proof/img.png)]

## git pull --rebase

### Syntax
```bash
git pull --rebase
```

### Purpose
Fetches remote changes and rebases local commits on top of them to maintain a linear history.

### Example
```bash
git pull --rebase origin main
```

![[git pull rebase](./proof/img.png(69).png)]

## git push

### Syntax
```bash
git push
```

### Purpose
Uploads local commits to the remote repository.

### Example
```bash
git push origin main
```

![[git push](./proof/img.png)]

---

## git push -u origin branch-name

### Syntax
```bash
git push -u origin feature
```

### Purpose
Pushes a branch to the remote repository and sets the upstream tracking branch.

### Example
```bash
git push -u origin feature
```

![[git push upstream](./proof/img.png)]

## git push --force

### Syntax
```bash
git push --force
```
```

```
### Purpose
Forces Git to overwrite the remote repository history with local changes.

### Example
```bash
git push --force origin main
```

![git push force](./proof/img.png)

# 9. Stash Commands
## git stash

### Syntax
```bash
git stash
```

### Purpose
Temporarily saves uncommitted changes so the working directory becomes clean.

### Example
```bash
git stash
```

![git stash](./proof/img.png)

## git stash list

### Syntax
```bash
git stash list
```

### Purpose
Displays all stashed changes.

### Example
```bash
git stash list
```

![git stash list](./proof/img.png)

## git stash pop

### Syntax
```bash
git stash pop
```

### Purpose
Applies the latest stash and removes it from the stash list.

### Example
```bash
git stash pop
```

![git stash pop](./proof/img.png)

## git stash apply

### Syntax
```bash
git stash apply
```

### Purpose
Applies a stash without removing it from the stash list.

### Example
```bash
```

```

git stash apply
```
![[git stash apply](./proof/img.png)]

## git stash drop

### Syntax
```bash
git stash drop stash@{0}
```

### Purpose
Deletes a specific stash from the stash list.

### Example
```bash
git stash drop stash@{0}
```
![[git stash drop](./proof/img.png)]

## git stash clear

### Syntax
```bash
git stash clear
```

### Purpose
Deletes all saved stashes.

### Example
```bash
git stash clear
```
![[git stash clear](./proof/img.png)]

# 10. Reset & Undo Commands

## git reset
### Syntax
```bash
git reset HEAD~1
```

### Purpose
Moves the current branch pointer to a previous commit.

### Example
```bash
git reset HEAD~1
```
![[git reset](./proof/img.png(79).png)]

## git reset --soft

### Syntax
```bash
git reset --soft HEAD~1
```

### Purpose
Moves HEAD to a previous commit but keeps changes staged.

### Example
```bash
git reset --soft HEAD~1
```
![[git reset soft](./proof/img.png(80).png)]

## git reset --mixed

```

```
### Syntax
```bash
git reset --mixed HEAD~1
```

### Purpose
Moves HEAD to a previous commit and unstages changes.

### Example
```bash
git reset --mixed HEAD~1
```

![[git reset mixed](./proof/img.png(80).png>)]

## git reset --hard

### Syntax
```bash
git reset --hard HEAD~1
```

### Purpose
Moves HEAD to a previous commit and deletes all changes in the working directory.

### Example
```bash
git reset --hard HEAD~1
```

![[git reset hard](./proof/img.png(81).png>)]

## git revert

### Syntax
```bash
git revert HEAD
```

### Purpose
Creates a new commit that reverses the changes introduced by a previous commit.

### Example
```bash
git revert HEAD
```

![[git revert](./proof/img.png(82).png>)]

## git clean -f

### Syntax
```bash
git clean -f
```

### Purpose
Removes untracked files from the working directory.

### Example
```bash
git clean -f
```

![[git clean f](./proof/img.png)]

## git clean -fd

### Syntax
```bash
git clean -fd
```

### Purpose
```


Removes untracked files and directories.

Example

```
```bash
```

```
git clean -fd
```

```
```
```

```
![git clean fd](./proof/img.png)
```

11. Rebasing Commands

git rebase

Syntax

```
```bash
```

```
git rebase <branch-name>
```

```
```
```

Purpose

Reapplies commits from the current branch on top of another branch, creating a cleaner and linear commit history.

Example

```
```bash
```

```
git rebase main
```

```
```
```

```
![git rebase](./proof/img.png%20(84).png)
```

git rebase -i

Syntax

```
```bash
```

```
git rebase -i HEAD~3
```

```
```
```

Purpose

Starts an interactive rebase that allows you to edit, reorder, squash, or remove commits.

Example

```
```bash
```

```
git rebase -i HEAD~3
```

```
```
```

```
![git rebase interactive](./proof/img.png%20(84).png)
```

git rebase --continue

Syntax

```
```bash
```

```
git rebase --continue
```

```
```
```

Purpose

Continues the rebasing process after resolving conflicts.

Example

```
```bash
```

```
git add file.txt
```

```
git rebase --continue
```

```
```
```

```
![git rebase continue](./proof/img.png%20(84).png)
```

git rebase --abort

Syntax

```
```bash
```

```
git rebase --abort
```

```
```
```

Purpose

Stops the rebase process and returns the repository to the state before the rebase started.

Example

```
```bash
```

```
git rebase --abort
```

```
```
```

```
![git rebase abort](./proof/img.png)
```

12. Cherry Pick & Patch Commands

```

## git cherry-pick
### Syntax
```bash
git cherry-pick <commit-id>
```

### Purpose
Applies a specific commit from another branch to the current branch.

### Example
```bash
git cherry-pick a1b2c3d
```

![[git cherry pick](./proof/img.png%20(86).png)]

## git format-patch
### Syntax
```bash
git format-patch -1 HEAD
```

### Purpose
Creates a patch file from commits that can be shared or applied to another repository.

### Example
```bash
git format-patch -1 HEAD
```

![[git format patch](./proof/img.png)]

## git apply
### Syntax
```bash
git apply <patch-file>
```

### Purpose
Applies the changes from a patch file to the working directory.

### Example
```bash
git apply 0001-feature-commit.patch
```

![[git apply](./proof/img.png)]

## git am
### Syntax
```bash
git am <patch-file>
```

### Purpose
Applies a patch and automatically creates a commit with the patch's commit message.

### Example
```bash
git am 0001-feature-commit.patch
```

![[git am](./proof/img.png%20(89).png)]

# 13. Tagging Commands

## git tag
### Syntax
```bash
git tag
```

### Purpose
Lists all tags in the repository.

### Example
```bash
git tag
```

```

```

![git tag](./proof/img.png)

## git tag -a
### Syntax
```bash
git tag -a v1.0 -m "First release"
```

### Purpose
Creates an annotated tag with a message.
### Example
```bash
git tag -a v1.0 -m "First release"
```

![git tag annotated](./proof/img.png)

## git tag -d
### Syntax
```bash
git tag -d v1.0
```

### Purpose
Deletes a tag from the local repository.
### Example
```bash
git tag -d v1.0
```

![git tag delete](./proof/img.png)

## git push origin --tags
### Syntax
```bash
git push origin --tags
```

### Purpose
Pushes all local tags to the remote repository.
### Example
```bash
git push origin --tags
```

![git push tags](./proof/img.png)

# 14. Submodule Commands

## git submodule add
### Syntax
```bash
git submodule add <repository-url>
```

### Purpose
Adds another Git repository as a submodule inside the current repository.
### Example
```bash
git submodule add https://github.com/git/git.git
```

![git submodule add](./proof/img.png)

## git submodule init
### Syntax
```bash
git submodule init
```

### Purpose
Initializes the submodule configuration defined in the repository.
### Example
```bash
git submodule init

```

```

...
![git submodule init](./proof/img.png)

git submodule update
Syntax
```bash
git submodule update
```

Purpose
Fetches and checks out the correct version of the submodule.
Example
```bash
git submodule update
```

![git submodule update](./proof/img.png)

15. Debugging Commands

git bisect
Syntax
```bash
git bisect
```

Purpose
Helps find the commit that introduced a bug by performing a binary search through commit history.
Example
git bisect
![git bisect](./proof/img.png)

git bisect start
Syntax
```bash
git bisect start
```

Purpose
Starts the bisect process to locate a faulty commit.
Example
```bash
git bisect start
```

![git bisect start](./proof/img.png)

git bisect good
Syntax
```bash
git bisect good <commit-id>
```

Purpose
Marks a commit where the bug did not exist.
Example
```bash
git bisect good a1b2c3d
```

![git bisect good](./proof/img.png)

git bisect bad
Syntax
```bash
git bisect bad
```

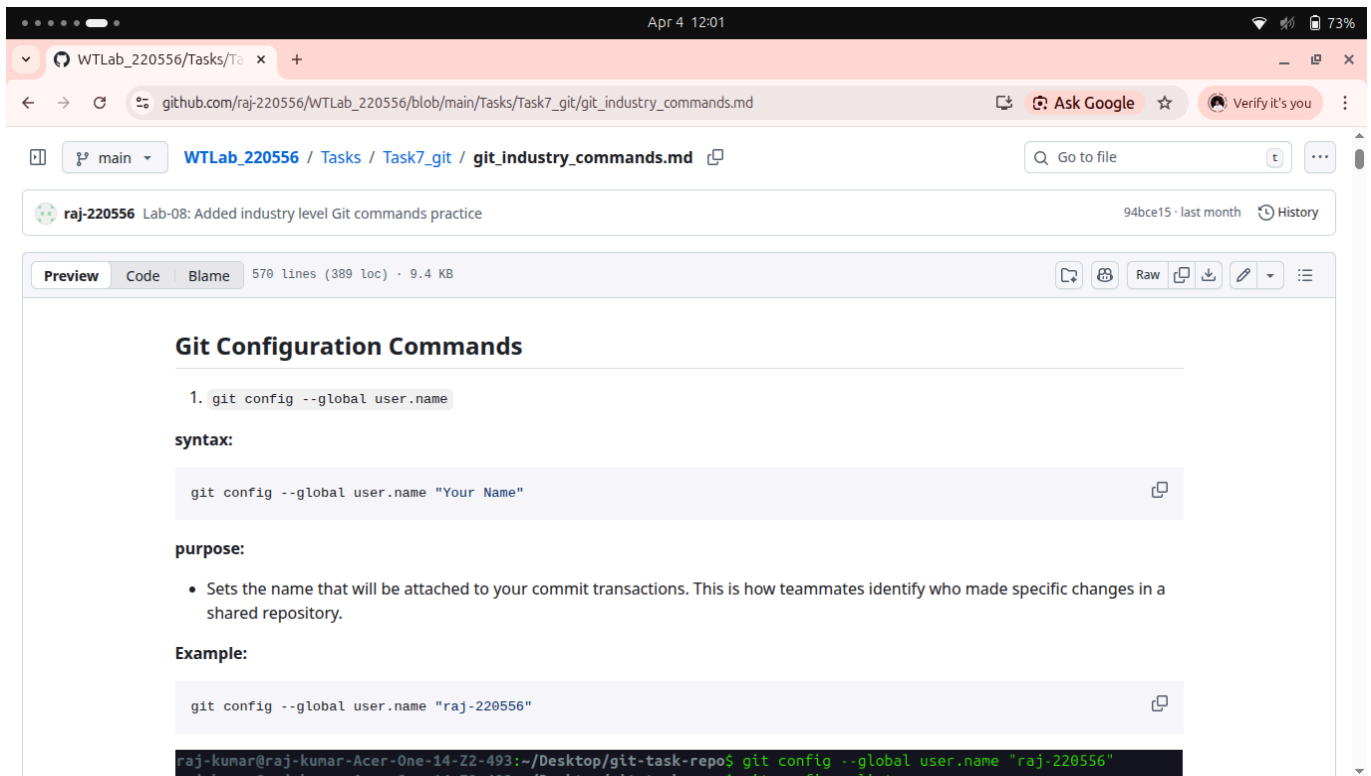
Purpose
Marks the current commit as containing the bug.
Example
```bash
git bisect bad

```

```
...
```

```
![git bisect bad](./proof/img.png)
```

Results / Screenshots

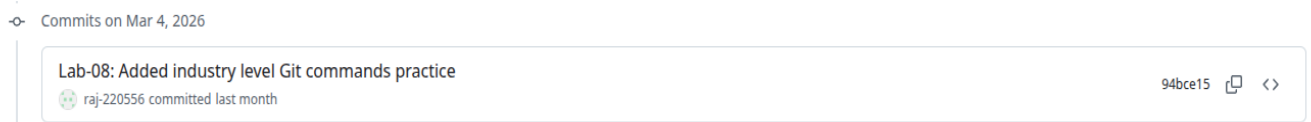


GitHub Repository Evidence:

Repository Details:

- **GitHub User ID:** raj-220556
- **Repository Name:** WTLab_220556
- **Repository URL:** https://github.com/raj-220556/WTLab_220556.git

Required Screenshots:



Conclusion:

Successfully executed and documented Industry-Level Git commands and integrated GitHub collaboration tools. Demonstrated the ability to manage complex repository states, resolve conflicts, and maintain professional documentation within the SudoSchool project.

Student Details :

Name: G. Raj Kumar
Roll Number: 31
Section: CSE-2
Date: 2026-03-31

Faculty Remarks :

Lab 9: JQuery

Aim

To implement and demonstrate core jQuery fundamentals—including DOM manipulation, event handling, traversal, and advanced effects—while maintaining a structured development workflow using Git and GitLab.

Description

This experiment focuses on utilizing the jQuery library to simplify client-side scripting. Key concepts implemented include various selectors (Basic, Hierarchy, Filter, Attribute), attribute management (attr, val), and comprehensive event handling (Mouse, Keyboard, Form). The project also explores DOM Traversal to navigate the HTML tree and jQuery Effects to create interactive UI animations. All progress is managed via a GitLab-based version control system.

Task:

Develop a multi-page web application for SudoSchool that demonstrates:

- **Selectors & Attributes:** Implementing dynamic content changes and form interactions.
- **Event System:** Handling user inputs through mouse clicks, keyboard presses, and form submissions.
- **Style & Traversal:** Toggling CSS classes and navigating parent-child relationships in the DOM.
- **UI Effects:** Implementing fades, slides, and custom animations with sequence control.
- **GitLab Workflow:** Initializing a local repository, connecting to a GitLab remote, and pushing progressive commits.

Definitions:

- **jQuery Selector:** A function used to find or select HTML elements based on their name, ID, classes, types, attributes, and more.
- **DOM Traversal:** The process of moving through the DOM tree to find related elements based on their relationship to the current element.
- **Event Bubbling:** A concept where an event triggers on the deepest target element and then successively triggers on its ancestors.
- **Chaining:** A jQuery feature that allows multiple methods to be run on the same element(s) within a single statement.

Code / Implementation:

script.js:

```
$(document).ready(function() {
  // --- TASK 1: SELECTORS ---
  $("#btn-apply-selectors").click(function() {
    // Basic & Hierarchy
    $("div#selector-demo p").css("color", "blue");
    $("ul.list-items > li").css("border-left", "5px solid green");

    // Filter Selectors
    $("ul.list-items li:first").css("background", "#eee");
    $("ul.list-items li:even").addClass("highlight");

    // Attribute Selectors
    $("input[name='user']").css("border", "2px solid red");
  });
  // --- TASK 2: ATTRIBUTES ---
  $("#btn-attr-demo").click(function() {
    // attr() & val()
    $("#demo-img").attr("src", "../assets/images/samplelogo2.png");
    let currentVal = $("#attr-input").val();
    alert("Input Value is: " + currentVal);

    // prop() & removeAttr()
    $("#demo-check").prop("checked", true);
    $("#attr-input").removeAttr("placeholder");
  });
});
```

```

});
// --- TASK 3: EVENTS ---
$("#click-btn").click(function() { alert("Button Clicked!"); });
$("#click-btn").dblclick(function() { $(this).text("Double Clicked!"); });

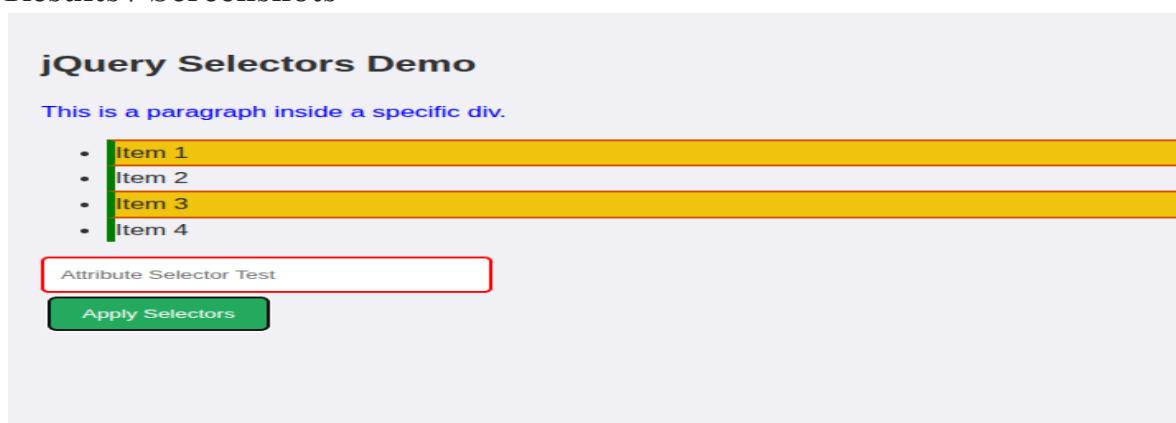
$("#hover-box").hover(
    function() { $(this).css("background", "red").text("In!"); },
    function() { $(this).css("background", "#3498db").text("Out!"); }
);
$("#key-input").keyup(function() {
    $("#key-status").text("Typing: " + $(this).val());
});
$("#lab-form").on("submit", function(e) {
    e.preventDefault();
    alert("Form submitted safely!");
});
// --- TASK 4: STYLE MANIPULATION ---
$("#btn-add-class").click(function() { $("#style-text").addClass("highlight"); });
$("#btn-remove-class").click(function() { $("#style-text").removeClass("highlight"); });
$("#btn-toggle-theme").click(function() { $("body").toggleClass("dark-theme"); });
// --- TASK 5: TRAVERSING ---
$("#btn-traverse").click(function() {
    // Parent/Child
    $("span").parent().css({"color": "red", "border": "2px solid red"});
    $(".ancestor").find("span").css("background", "yellow");

    // Siblings
    $("h2").next().css("text-decoration", "underline");
});
// --- TASK 6 & 7: EFFECTS & ANIMATION ---
$("#btn-show-hide").click(function() { $(".box-fx").toggle(500); });
$("#btn-fade").click(function() { $(".box-fx").fadeToggle(1000); });
$("#btn-slide").click(function() { $(".box-fx").slideToggle(); });
$("#btn-animate").click(function() {
    $(".box-fx").animate({
        left: '250px',
        opacity: '0.5',
        height: '150px',
        width: '150px'
    }, 2000).delay(500).animate({
        left: '0px',
        opacity: '1',
        height: '120px',
        width: '120px'
    });
});

$("#btn-stop").click(function() {
    $(".box-fx").stop(true, true);
});
});

```

Results / Screenshots



Style Manipulation

Change my style using jQuery classes.

Add Highlight

Remove Highlight

Toggle Dark Theme

DOM Traversing

div (great-grandparent).

ul (grandparent).

li (direct parent).

span (child).

Traverse Up/Down

jQuery Effects & Animation Control

Toggle Show/Hide

Fade Toggle

Slide Toggle

Run Animation Sequence

Stop Animation

FX Target

GitHub Repository Evidence:

Repository Details:

- **GitHub User ID:** raj-220556
- **Repository Name:** WTLab_220556
- **Repository URL:** https://github.com/raj-220556/WTLab_220556.git

Required Screenshots:

Commits on Apr 4, 2026

Lab-10: JQuery Selectors/Events/traversing/Effects

raj-220556 committed 4 minutes ago

719aefb

**Conclusion**

Successfully implemented jQuery DOM manipulation, events, and effects within the SudoSchool project. Demonstrated proficiency in navigating the DOM tree and controlling animations while maintaining a professional GitLab workflow with regular commits and remote synchronization.

Student Details :

Name: G. Raj Kumar

Roll Number: 31

Section: CSE-2

Date: 2026-03-31

Faculty Remarks :

Lab 10: Bootstrap

Aim:

To implement and demonstrate a fully responsive Digital Resume / Portfolio using the Bootstrap 5 framework and jQuery, while managing the development lifecycle through Git and GitHub.

Description:

This experiment focuses on the practical application of the Bootstrap 5 Grid System and its extensive component library to build a mobile-first personal website. Key concepts include responsive breakpoints (sm to xxl), navigation components, form validation, and interactive JavaScript elements like Modals and Carousels. Additionally, it involves integrating jQuery for dynamic DOM interactions and analyzing real-world Bootstrap implementations through website exploration.

Task:

Develop a web application for SudoSchool that demonstrates the following:

- **Responsive Layout:** Utilizing the 12-column grid system with containers, rows, and nested columns.
- **UI Components:** Implementing Cards for projects, Progress Bars for skills, and a responsive Navbar with a collapse menu.
- **Interactive Elements:** Using Bootstrap JavaScript components (Modals, Tooltips, Popovers) and jQuery event handling.
- **Forms & Validation:** Creating a complete contact form with various input types and validation states.
- **Website Analysis:** Exploring five external websites to identify and document Bootstrap-specific class usage.

Definitions:

- **Breakpoints:** Customizable widths that determine how your responsive layout behaves across different device or viewport sizes (e.g., `md` for tablets).
- **Container-fluid:** A Bootstrap class that provides a full-width container, spanning the entire width of the viewport.
- **Modal:** A lightweight, multi-purpose JavaScript popup that is desirable for user alerts or to display detailed project information without leaving the page.
- **Gutter:** The padding between your columns, used to responsively space and align content in the Bootstrap grid system.

Code / Implementation:

resume.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>G. Raj Kumar | Cybersecurity & Web Dev</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap-icons/font/bootstrap-icons.css"
rel="stylesheet">
</head>
<body>
<nav class="navbar navbar-expand-lg navbar-dark bg-dark sticky-top">
  <div class="container">
    <a class="navbar-brand" href="#">RAJ KUMAR</a>
    <button class="navbar-toggler" data-bs-toggle="collapse" data-bs-target="#navMenu">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navMenu">
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link" href="#home">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="#about">About</a></li>
```

```

        <li class="nav-item"><a class="nav-link" href="#skills">Skills</a></li>
        <li class="nav-item"><a class="nav-link" href="#projects">Projects</a></li>
        <li class="nav-item"><a class="nav-link" href="#contact">Contact</a></li>
    </ul>
</div>
</div>
</nav>
<section id="home" class="text-center p-5">
    <div class="container py-5">
        <h1 class="display-3">Hi, I'm G. Raj Kumar</h1>
        <p class="lead">B.Tech CSE Student | Cybersecurity Enthusiast | Full Stack Developer</p>
        <div class="mt-4">
            <a href="https://z3ncrypt3dbu9.vercel.app/" target="_blank" class="btn
btn-outline-light me-2">Visit Portfolio</a>
            <button class="btn btn-light"><i class="bi bi-download"></i> Download CV</button>
        </div>
    </div>
</section>
<section id="about" class="p-5">
    <div class="container">
        <div class="row align-items-center">
            <div class="col-md-5 text-center">
                
            </div>
            <div class="col-md-7">
                <h2 class="mb-4">About Me</h2>
                <p>I am a Computer Science Engineering student at RGUKT with a deep passion for
<strong>Cybersecurity automation</strong> and <strong>Web Technologies</strong>. I specialize
in building secure applications and analyzing network traffic using Python.</p>
                <p>Currently focused on SOC analysis and developing AI-driven security solutions.</p>
                <div class="alert alert-info border-0 shadow-sm">
                    <i class="bi bi-geo-alt-fill"></i> Based in Jaggaiahpet, Andhra Pradesh
                </div>
            </div>
        </div>
    </div>
</section>
<section id="skills" class="p-5 bg-light">
    <div class="container">
        <h2 class="text-center mb-5">Technical Proficiency</h2>
        <div class="row">
            <div class="col-md-6">
                <h5>Development</h5>
                <div class="mb-3">
                    <span>HTML/CSS/JS & Bootstrap</span>
                    <div class="progress"><div class="progress-bar bg-primary" style="width:
90%">90%</div></div>
                </div>
                <div class="mb-3">
                    <span>PHP & MySQL</span>
                    <div class="progress"><div class="progress-bar bg-success" style="width:
75%">75%</div></div>
                </div>
            </div>
            <div class="col-md-6">
                <h5>Security & Scripting</h5>
                <div class="mb-3">
                    <span>Python (Scapy, Pandas)</span>
                    <div class="progress"><div class="progress-bar bg-warning" style="width:
85%">85%</div></div>
                </div>
                <div class="mb-3">
                    <span>Bash & Linux (Ubuntu)</span>
                    <div class="progress"><div class="progress-bar bg-dark" style="width:
80%">80%</div></div>
            </div>
        </div>
    </div>

```

```

        </div>
    </div>
</div>
</div>
</section>
<section id="projects" class="p-5">
    <div class="container">
        <h2 class="text-center mb-5">Featured Projects</h2>
        <div class="row g-4">
            <div class="col-md-6 col-lg-4">
                <div class="card h-100 shadow-sm">
                    <div class="card-body">
                        <h5 class="card-title text-primary">ASOC Project</h5>
                        <p class="card-text">Adaptive AI-Driven Security Operations Center. Integrating AI
for automated threat detection and response.</p>
                        <span class="badge bg-secondary">Python</span> <span class="badge
bg-secondary">AI</span>
                    </div>
                </div>
            </div>
            <div class="col-md-6 col-lg-4">
                <div class="card h-100 shadow-sm">
                    <div class="card-body">
                        <h5 class="card-title text-primary">SudoSchool</h5>
                        <p class="card-text">A specialized educational platform project developed as part
of web technology lab tasks.</p>
                        <span class="badge bg-secondary">PHP</span> <span class="badge
bg-secondary">MySQL</span>
                    </div>
                </div>
            </div>
            <div class="col-md-6 col-lg-4">
                <div class="card h-100 shadow-sm">
                    <div class="card-body">
                        <h5 class="card-title text-primary">Mini Jira / Trello</h5>
                        <p class="card-text">A collaborative project management tool built with a focus on
task tracking and team workflow.</p>
                        <span class="badge bg-secondary">JavaScript</span> <span class="badge
bg-secondary">Bootstrap</span>
                    </div>
                </div>
            </div>
        </div>
    </div>
</section>
<section class="p-5 bg-light">
    <div class="container">
        <h2 class="mb-4">Education & Internships</h2>
        <div class="table-responsive shadow-sm bg-white rounded">
            <table class="table table-hover mb-0">
                <thead class="table-dark">
                    <tr>
                        <th>Year</th>
                        <th>Program / Internship</th>
                        <th>Organization</th>
                    </tr>
                </thead>
                <tbody>
                    <tr>
                        <td>2022 - Present</td>
                        <td>B.Tech Computer Science (CSE)</td>
                        <td>RGUKT (ID: N220556)</td>
                    </tr>
                    <tr>
                        <td>March 2026</td>
                        <td>Cybersecurity Virtual Internship</td>
                    </tr>
                </tbody>
            </table>
        </div>
    </div>
</section>

```

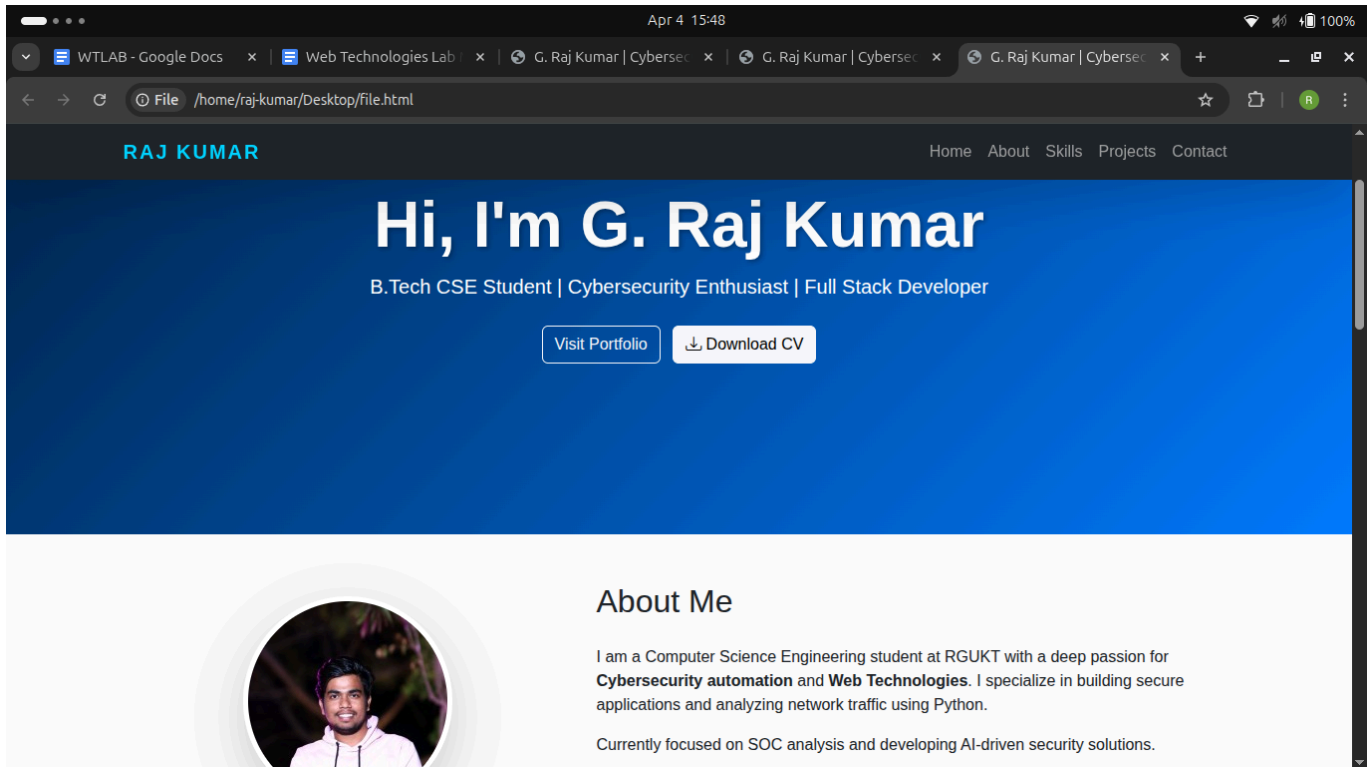
```

        <td>VOIS & Vodafone Idea Foundation</td>
    </tr>
</tbody>
</table>
</div>
</div>
</section>
<section id="contact" class="p-5">
    <div class="container">
        <h2 class="text-center mb-5">Get In Touch</h2>
        <div class="row justify-content-center">
            <div class="col-md-8">
                <form class="shadow p-4 rounded bg-white">
                    <div class="row">
                        <div class="col-md-6 mb-3">
                            <label class="form-label">Full Name</label>
                            <input type="text" class="form-control" placeholder="Your Name">
                        </div>
                        <div class="col-md-6 mb-3">
                            <label class="form-label">Email Address</label>
                            <input type="email" class="form-control" placeholder="n220556@rguktn.ac.in">
                        </div>
                    </div>
                    <div class="mb-3">
                        <label class="form-label">Subject</label>
                        <select class="form-select">
                            <option selected>Hiring / Collaboration</option>
                            <option>Project Discussion</option>
                            <option>Others</option>
                        </select>
                    </div>
                    <div class="mb-3">
                        <label class="form-label">Message</label>
                        <textarea class="form-control" rows="4" placeholder="How can I help
you?"></textarea>
                    </div>
                    <button class="btn btn-primary w-100 py-2">Send Message</button>
                </form>
            </div>
        </div>
    </div>
</section>
<footer class="bg-dark text-white text-center p-4">
    <div class="container">
        <p class="mb-2">© 2026 G. Raj Kumar</p>
        <div class="fs-4">
            <a href="https://github.com/raj-220556" target="_blank" class="text-white mx-2"><i
class="bi bi-github"></i></a>
            <a href="#" class="text-white mx-2"><i class="bi bi-linkedin"></i></a>
            <a href="#" class="text-white mx-2"><i class="bi bi-twitter-x"></i></a>
        </div>
    </div>
</footer>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
<script>
$(document).ready(function(){
    $(window).scroll(function() {
        if ($(this).scrollTop() > 50) {
            $(' .navbar').addClass('shadow-lg bg-dark');
        } else {
            $(' .navbar').removeClass('shadow-lg');
        }
    });
});
});

```

```
</script>
</body>
</html>
```

Results / Screenshots:

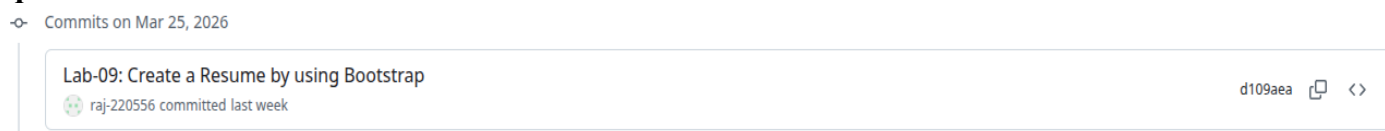


GitHub Repository Evidence:

Repository Details:

- **GitHub User ID:** raj-220556
- **Repository Name:** WTLab_220556
- **Repository URL:** https://github.com/raj-220556/WTLab_220556.git

Required Screenshots:



Conclusion

Successfully implemented a fully responsive Digital Resume using Bootstrap 5 and jQuery. Demonstrated mastery of the grid system, CSS/JS components, and form validation while ensuring cross-device compatibility and maintaining a professional Git-based version control history.

Student Details :

Name: G. Raj Kumar
Roll Number: 31
Section: CSE-2
Date: 2026-03-31

Faculty Remarks :